

A HUMAN-INSPIRED READING AGENT WITH GIST MEMORY OF VERY LONG CONTEXTS

Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, Ian Fischer

Google DeepMind

{leekh, xinyunchen, hirokifuruta, canny, iansf}@google.com

Project Website and Demo: read-agent.github.io

ABSTRACT

Current Large Language Models (LLMs) are not only limited to some maximum context length, but also are not able to robustly consume long inputs. To address these limitations, we propose ReadAgent, an LLM agent system that increases effective context length up to $20\times$ in our experiments. Inspired by how humans interactively read long documents, we implement ReadAgent as a simple prompting system that uses the advanced language capabilities of LLMs to (1) decide what content to store together in a memory episode, (2) compress those memory episodes into short episodic memories called *gist memories*, and (3) take actions to look up passages in the original text if ReadAgent needs to remind itself of relevant details to complete a task. We evaluate ReadAgent against baselines using retrieval methods, using the original long contexts, and using the gist memories. These evaluations are performed on three long-document reading comprehension tasks: QuALITY, NarrativeQA, and QMSum. ReadAgent outperforms the baselines on all three tasks while extending the effective context window by $3.5 - 20\times$.

1 INTRODUCTION

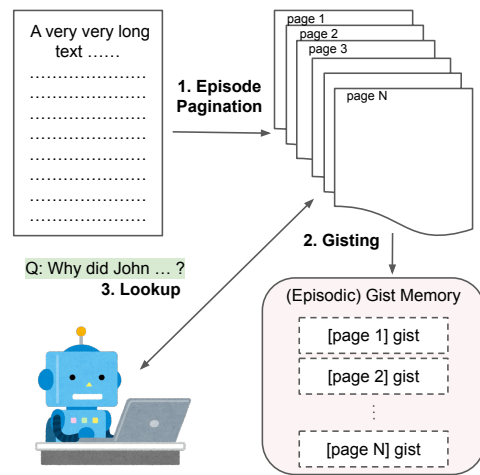


Figure 1: ReadAgent workflow.

Transformer-based Large Language Models (LLMs) are highly capable of language understanding, but the amount of text that LLMs are able to read at one time is constrained. Not only is there an explicit context length limitation, but it has also been found that performance of LLMs tends to decline with increasingly long inputs even when they don't actually exceed the explicit context window Liu et al. (2023); Shi et al. (2023). In contrast, humans can read, understand, and reason over very long texts, such as a series of interrelated books.

We posit that an underlying reason for this gap is inherent in the differences in reading approaches. Typically, we use LLMs to consume the exact given content word-by-word and the process is relatively passive. On the other hand, humans read and reason over long text differently. First, the exact information tends to be forgotten quickly, whereas the fuzzier gist information, i.e.

the substance irrespective of exact words, from past readings lasts much longer Reyna & Brainerd (1995b;a); Reyna (2012)¹. Second, human reading is an interactive process. When we need to remind

¹Fuzzy-trace theory Reyna & Brainerd (1995b) posits that people form two types of memory representations about a past event – verbatim and gist memories. Gist memories, often episodic, are fuzzy memories of past events, whereas verbatim memories contain details of past events. People prefer to reason with gists rather than with verbatim memories Reyna (2008).

一种具备长上下文要点记忆、受人类启发的阅读智能体

Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, Ian Fischer

Google DeepMind {leekh, xinyunchen, hirokifuruta, canny, iansf}@google.com

项目网站与演示: read-agent.github.io

摘要

当前的大语言模型不仅受限于最大上下文长度，而且无法稳健地处理长输入。为应对这些限制，我们提出了ReadAgent，这是一个LLM智能体系统，在我们的实验中，它有效上下文长度提升至 $20\times$ 。受人类交互式阅读长文档方式的启发，我们将ReadAgent实现为一个简单的提示系统，它利用大语言模型的高级语言能力来：(1) 决定将哪些内容一起存储在一个记忆片段中，(2) 将这些记忆片段压缩成称为要点记忆的简短情景记忆，以及 (3) 在需要回忆相关细节以完成任务时，采取行动查找原始文本中的段落。我们使用检索方法、原始长上下文以及要点记忆，将ReadAgent与基线进行比较评估。这些评估在三个长文档阅读理解任务上进行：QuALITY、NarrativeQA和QMSum。ReadAgent在所有三个任务上均优于基线，同时将有效上下文窗口扩展了 $3.5 - 20\times$ 。

1 引言

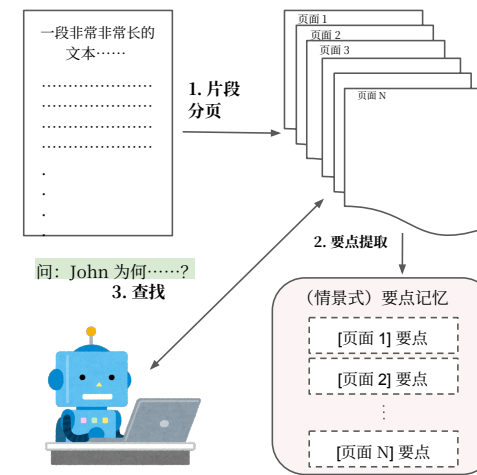


图1: ReadAgent工作流程。

基于Transformer的大语言模型在语言理解方面能力很强，但大语言模型一次能够阅读的文本量是受限的。这不仅存在明确的上下文长度限制，而且研究发现，即使输入文本并未实际超出明确的上下文窗口，大语言模型的性能也往往会随着输入长度的增加而下降 (Liu等人 (2023); Shi等人 (2023))。相比之下，人类能够阅读、理解并推理非常长的文本，例如一系列相互关联的书籍。

我们认为，造成这种差距的一个根本原因在于阅读方式的固有差异。通常，我们使用大语言模型来逐字逐句地消化给定的确切内容，这个过程相对被动。另一方面，人类阅读和推理长文本的方式则不同。首先，确切的信息往往很快被遗忘，而较为模糊的要点信息，即

无论具体措辞如何，从过往阅读中留存下来的实质内容（要义）持续时间要长得多 (Reyna & Brainerd, 1995b;a; Reyna, 2012)¹。其次，人类的阅读是一个互动过程。当我们需要回忆

¹模糊痕迹理论 Reyna & Brainerd (1995b) 认为，人们对过去事件会形成两种类型的记忆表征——逐字记忆和要点记忆。要点记忆通常是情景性的，是对过去事件的模糊记忆，而逐字记忆则包含过去事件的细节。人们更倾向于使用要点而非逐字记忆进行推理 Reyna(2008)。

ourselves of relevant details in order to complete a task, such as answering a question, we look them up in the original text.

We think that using the fuzzy gist memory to capture global context and attending to local details together enables humans to reason over very long context efficiently, in terms of how much information to process at once, and is also important for comprehension. For example, if we were to infer the intention of a fictional character’s specific action described on a page in a novel, besides focusing on the surrounding pages, we likely also need to understand the overall story and the character’s personality from reading the whole book (see Appendix C for more analysis).

Motivated by these observations, we propose ReadAgent, an LLM agent system that handles long content inspired by the human approach. ReadAgent is simple to implement and can be built entirely by prompting a previously-trained LLM. As illustrated in Figure 1, it takes three primary steps: **(1) episode pagination**, where we prompt the LLM to decide where to pause in reading contiguous text; the content between pause points becomes an episode, which we refer to as *pages* in this work; **(2) memory gisting**, where we prompt the LLM to compress each page into a shorter *gist* and associate the gist with a corresponding context (e.g. which page the gist was from) – this gives the episodic *gist memory*; **(3) interactive look-up**, where the LLM looks at the given task and the complete set of gists in-context, makes decision on what page(s) to look up, combines the gists with these raw pages, and solves the task.

We evaluate ReadAgent by comparing against using only the gist memory without interactive look-up, using full text for datasets that can fit in the context window, and using retrieval methods to look up pages. ReadAgent outperforms all baselines across three challenging long-document comprehension tasks – QuALITY, NarrativeQA and QMSum – while increasing the effective context length significantly compared to the original LLM. On NarrativeQA Gutenberg test set, whose average length is 71k words and whose maximum is 343k words, ReadAgent improves the LLM rating (Section 4.1) by 12.97% and ROUGE-L by 31.98% over the best retrieval baseline and increases the effective context length by $\sim 20\times$. On QuALITY, where the articles can fit in an 8K context window, ReadAgent outperforms using full text with a $3.5\times$ effective context length.

Finally, in Appendix D, we adapt ReadAgent to web navigation, which is a fundamentally very-long context agent setting. We find that ReadAgent is simple to adapt to this setting and shows promising performance.

Our primary contributions are:

- **ReadAgent**, our human-inspired LLM agent that generates gist memories and looks up information as needed for solving tasks on long contexts (Section 3).
- Demonstration of significant performance advantages and scalability through a comprehensive experimental evaluation on challenging long-context benchmarks, comparisons against popular baselines, and analysis (Section 4).

2 RELATED WORK

Long-Context LLMs The most direct way to improve LLM long-context performance is to train or fine-tune LLMs with longer context windows (Beltagy et al., 2020; Zaheer et al., 2020; Guo et al., 2022; Ainslie et al., 2023; Tay et al., 2022; Chen et al., 2023c). Another approach is to explore new architectures or efficient implementations of the Transformer (Vaswani et al., 2017) attention layers to reduce the need of long-context fine-tuning (Chen et al., 2023b; Press et al., 2022; Xiao et al., 2023; Jin et al., 2024; Han et al., 2023). However, LLM performance tends to decline with increasingly long inputs even when they don’t exceed the specified context length Liu et al. (2023). LLM performance is also shown to be sensitive to distracting information in the context Shi et al. (2023). Thus, the effective context length could be shorter than the explicit limit. Our approach is complimentary to these approaches, scaling the effective context length of the underlying model while reducing the amount of distracting information in context, and requiring neither architectural changes nor training.

Retrieval Retrieval Augmented Generation (RAG) techniques (Chen et al., 2017; Dinan et al., 2019; Lewis et al., 2020; Izacard & Grave, 2021; Wu et al., 2022; Park et al., 2023; Zhong et al., 2023) allow an LLM to query task-relevant information from a large database of documents or document pieces.

相关细节以完成任务时，例如回答一个问题，我们会去原始文本中查找它们。

我们认为，利用模糊要点记忆来捕捉全局上下文，同时关注局部细节，使人类能够高效地对超长上下文进行推理——这涉及一次处理多少信息的问题——并且对理解也至关重要。例如，若要推断小说中某一页描述的虚构角色的特定行为意图，除了关注其周围的页面，我们很可能还需要通过阅读整本书来理解整体故事和角色的性格（更多分析请参见附录C）。

受这些观察启发，我们提出了ReadAgent，一个受人类方法启发、用于处理长内容的LLM智能体系统。ReadAgent实现简单，完全可以通过提示一个预训练的大语言模型来构建。如图1所示，它包含三个主要步骤：**(1) 片段分页**，即提示大语言模型决定在阅读连续文本时在何处暂停；暂停点之间的内容成为一个片段，我们在本工作中称之为页面；**(2) 记忆要点提取**，即提示大语言模型将每个页面压缩成更短的要点，并将该要点与相应的上下文（例如，该要点来自哪个页面）关联起来——这形成了片段式的要点记忆；**(3) 交互式查找**，即大语言模型查看给定任务和上下文中的完整要点集，决定要查找哪个（些）页面，将这些要点与原始页面结合，并解决任务。

我们通过以下方式评估ReadAgent：与仅使用要点记忆而不进行交互式查找的方法进行比较，对能放入上下文窗口的数据集使用全文，以及使用检索方法来查找页面。在三个具有挑战性的长文档理解任务——QuALITY、NarrativeQA和QMSum——中，ReadAgent的表现均优于所有基线，同时与原始大语言模型相比，显著增加了有效上下文长度。在平均长度为7.1万字、最大长度为34.3万字的NarrativeQA Gutenberg测试集上，ReadAgent相比最佳检索基线，将LLM评分（第4.1节）提升了12.97%，将ROUGE-L提升了31.98%，并将有效上下文长度增加了 $\sim 20\times$ 。在文章可以放入8千上下文窗口的QuALITY任务上，ReadAgent的表现优于使用全文的方法，其有效上下文长度为 $3.5\times$ 。

最后，在附录D中，我们将ReadAgent应用于网络导航，这是一个本质上具有超长上下文的智能体场景。我们发现ReadAgent能轻松适应此场景，并展现出有前景的性能。

我们的主要贡献包括：

- **ReadAgent**，我们受人类启发的、能生成要点记忆并根据需要在长上下文中查找信息以解决任务的大语言模型智能体（第3节）。
- 通过在具有挑战性的长上下文基准测试上全面的实验评估、与流行基线的比较以及分析，展示了显著的性能优势和可扩展性（第4节）。

2 相关工作

长上下文大语言模型 提升大语言模型长上下文性能最直接的方法是使用更长的上下文窗口来训练或微调大语言模型（Beltagy等人，2020；Zaheer等人，2020；Guo等人，2022；Ainslie等人，2023；Tay等人，2022；Chen等人，2023c）。另一种方法是探索新的架构或Transformer（Vaswani等人，2017）注意力层的高效实现，以减少对长上下文微调的需求（Chen等人，2023b；Press等人，2022；Xiao等人，2023；Jin等人，2024；Han等人，2023）。然而，即使输入长度未超过指定的上下文长度，大语言模型的性能也倾向于随着输入长度的增加而下降（Liu等人，2023）。研究还表明，大语言模型的性能对上下文中的干扰信息很敏感（Shi等人，2023）。因此，有效上下文长度可能比显式限制更短。我们的方法与这些方法是互补的，它在扩展底层模型有效上下文长度的同时，减少了上下文中的干扰信息量，并且既不需要架构更改，也不需要训练。

检索增强生成（RAG）技术（Chen等人，2017；Dinan等人，2019；Lewis等人，2020；Izacard与Grave，2021；Wu等人，2022；Park等人，2023；Zhong等人，2023）使得大语言模型能够从一个庞大的文档或文档片段数据库中查询与任务相关的信息。

Our work implements a form of retrieval by reasoning over a contextualized gist memory, all with zero-shot LLM prompting. This rethinking of retrieval directly leverages the strength and flexibility of LLM language understanding to reason about which documents to retrieve. Our approach is well-suited to densely-correlated long-document pieces, such as a series of books or a conversation history, but the database cannot scale arbitrarily, since the size of the gist memory is limited by the LLM’s context length, and the gist memory’s length correlates with the size of the database. In contrast, conventional retrieval approaches can handle larger databses than our approach. In this work, we compare against retrieval systems that use exactly the same set of documents as our approach.

LLM Agents for Long Texts LLMs can be used as agents to interactively handle very long texts. WebGPT Nakano et al. (2021) and WebShop Yao et al. (2022) learn browsing actions to search for the requested answer on the internet, despite not being designed to understand long documents. The PEARL Sun et al. (2023) system proposes action plans for better long-document comprehension through iterative prompting; Yuan et al. (2020) explicitly learns RL agents for similar purposes. Self-note Lanchantin et al. (2023) amortizes reasoning steps and interleaves intermediate notes with the original documents to improve reasoning. Yang et al. (2022) generates long outputs through iterative reasoning. However, these methods cannot address long input texts that exceed the LLM’s context length. Similar to this work, MemWalker Chen et al. (2023a) also reads long documents interactively through iterative prompting. It traverses a tree of different levels of summaries to search for task-related information. However, the hierarchical summary structure makes it difficult to reason over related but distant information at the same granularity (see Appendix G for more discussion).

3 READAGENT

Figure 1 shows an overview of ReadAgent, which we describe in detail below.

3.1 GIST MEMORY

A *gist memory* is an ordered collection of short gists of chunks of text from the original long context. Building a gist memory has two steps: *pagination* and *memory gisting*, described in turn below.

Episode Pagination When ReadAgent reads through a long text, it makes decisions on what content to store together in a memory episode by choosing where to pause reading. At each step, we provide the LLM some text that begins from the previous pause point and ends when it reaches a `max_words` limit. We prompt the LLM to choose which point between paragraphs would be a natural point to pause, and then treat the content between the previous and current pause points as an episode, which we also refer as a *page*. This is *episode pagination*, which we implement with the prompt in Figure 2.

As shown in the prompt, possible pause points are inserted between paragraphs as numbered tags (e.g. `<13>`), making this a multiple choice question for the LLM. We only start inserting these numbered tags after a `min_words` threshold to make sure that each page has at least `min_words`.

Memory Gisting For each *page*, we prompt the LLM to shorten the exact content into a *gist*, or summary, as shown in Figure 3.

We subsequently prepend a page tag to each gist (e.g. “`<Page 2>\n{GIST CONTENT}`”) to contextualize it (indicate where the gist was from), and then concatenate all gists. This gives us the gist memory.

You are given a passage that is taken from a larger text (article, book, ...) and some numbered labels between the paragraphs in the passage.

Numbered labels are in angle brackets. For example, if the label number is 19, it shows as `<19>` in text.

Please choose a label where it is natural to break reading.

The label can be a scene transition, the end of a dialogue, the end of an argument, a narrative transition, etc.

Please answer with the break point label and explain.

For example, if `<57>` is a good point to break, answer with “Break point: `<57>`\n Because ...”

Passage:

```
{...}
{PARAGRAPH 5 TEXT}
<5>
{PARAGRAPH 6 TEXT}
<6>
{PARAGRAPH 7 TEXT}
{...}
```

Figure 2: Example pagination prompt.

我们的工作通过基于情境化要点记忆进行推理，并以零样本大语言模型提示的方式，实现了一种基于推理的检索形式。这种对检索的重新思考直接利用了大语言模型语言理解的优势和灵活性，来推理应检索哪些文档。我们的方法非常适用于密集相关的长文档片段，例如一系列书籍或一段对话历史，但数据库的规模不能任意扩展，因为要点记忆的大小受限于大语言模型的上下文长度，且要点记忆的长度与数据库的规模相关。相比之下，传统检索方法能够处理比我们方法更大的数据库。在本工作中，我们与使用完全相同文档集的检索系统进行了比较。

面向长文本的大语言模型智能体大语言模型可以作为智能体来交互式地处理非常长的文本。

WebGPT (Nakano等人, 2021) 和 WebShop (Yao等人, 2022) 学习浏览操作以在互联网上搜索所请求的答案，尽管它们并非为理解长文档而设计。PEARL (Sun等人, 2023) 系统通过迭代提示提出行动计划以更好地进行长文档理解；Yuan等人 (2020) 则明确地学习强化学习智能体以达到类似目的。Self-note (Lanchantin等人, 2023) 分摊推理步骤，并将中间笔记与原始文档交错排列以改进推理。Yang等人 (2022) 通过迭代推理生成输出。然而，这些方法无法处理超出大语言模型上下文长度的长输入文本。与这项工作类似，MemWalker (Chen等人, 2023a) 也通过迭代提示交互式地阅读长文档。它遍历不同级别的摘要树来搜索任务相关信息。但是，这种分层摘要结构使得在同一粒度上对相关但距离较远的信息进行推理变得困难（更多讨论见附录G）。

3 READAGENT

图1展示了ReadAgent的概览，我们将在下文详细描述。

3.1 要点记忆

要点记忆是一个有序的集合，包含来自原始长文本块的简短要点。构建要点记忆有两个步骤：分页和记忆要点提取，下文将依次描述。

片段分页当ReadAgent阅读长文本时，它通过选择在何处暂停阅读来决定将哪些内容一起存储在一个记忆片段中。在每一步，我们向大语言模型提供一些文本，这些文本从上一个暂停点开始，并在达到最大词数限制时结束。我们提示大语言模型选择段落之间的哪个点是一个自然的暂停点，然后将

上一个和当前暂停点之间的内容视为一个片段，我们也称其为页面。这就是片段分页，我们使用图2中的提示来实现它。

如提示所示，可能的暂停点以编号标签（例如<13>）的形式插入在段落之间，这为LLM构成了一个多项选择题。我们仅在达到最小词数阈值后才开始插入这些编号标签，以确保每个页面至少包含最小词数。

记忆要点提取对于每个页面，我们提示大语言模型将确切内容缩短为一个要点或摘要，如图3所示。

随后，我们在每个要点前添加一个页面标签（例如“<页面 2>\n{要点内容}”）以提供上下文（指明该要点的来源），然后将所有要点连接起来。这就构成了我们的要点记忆。

你将获得一段取自更长文本（文章、书籍……）的段落，以及该段落中位于各段落之间的一些带编号的标签。编号标签用尖括号表示。例如，如果标签编号是19，它在文本中显示为 `<19>`。请选择一个适合中断阅读的自然断点标签。该标签可以是场景转换、对话结束、论点结束、叙述过渡等。请用断点标签和解释来回答。例如，如果 `<57>` 是一个合适的断点，请回答“断点: `<57>`\n 因为……” 段落: `{...}{段落5文本}{5}{段落6文本}{6}{ 7 } 段落文本 ... { }`

图2：分页提示示例。

We use the word “shorten” in the prompt to generate these summarizing gists as it tends to help preserve the narrative flow, making it more natural to concatenate. Using the word “summarize” tended to produce a re-structured summary in our experiments.

The original page size is a key factor for how compressed the gist is. Let’s say the smallest unit of text that we consider is a paragraph. Intuitively, a paragraph likely has some amount of mutual information with its neighbors. Thus, the larger chunk of text we group together, the more duplicated information we can remove. Empirically, compressing larger chunks of text with LLMs also tends to remove more details, which could affect performance. We control the page size by changing `min_words` and `max_words` in pagination. This trade-off is studied in Appendix A.

Please shorten the following passage.
Just give me a shortened version.
DO NOT explain your reason.
Passage:
{PAGE TEXT}

Figure 3: Example gisting prompt.

3.2 PARALLEL AND SEQUENTIAL INTERACTIVE LOOK-UP

For a given task about a long document, we want ReadAgent to take actions to look up relevant details in the original text in addition to using its gist memory. As the gist memories are contextualized with page numbers, we simply prompt the LLM to answer which page(s) it would like to look up and read again given the specific task. In the following we discuss two look-up strategies: looking up all pages at once in parallel (**ReadAgent-P**) and sequentially looking up one page at a time (**ReadAgent-S**).

ReadAgent-P As in the Figure 4 example for question-answering, typically we give it a maximum number of pages that it can look up but also instruct it to use as few pages as possible to avoid unnecessary computational overhead and distracting information. The following prompt shows parallel look-up, where the model requests multiple pages in response to a single prompt.

The selected raw pages replace the gist(s) at the corresponding positions in memory, preserving the overall narrative flow. Then we prompt the LLM again with the task and the updated memory and ask it to solve the task.

ReadAgent-S We also study the sequential look-up strategy (Figure 5), where the model requests one page at a time, up to some maximum number of pages. In sequential look-up, the model gets to see the previously expanded pages before deciding which page to expand. This gives the model access to more information than parallel look-up, so we might expect it to perform better in some situations. However, the larger number of interactions with the model increases the computational cost, so sequential look-up should only be used on tasks where it provides clear benefits.

3.3 COMPUTATIONAL TRADE-OFFS AND SCALABILITY

Episode pagination, memory gisting and interactive look-ups require iterative inference. As we show in the following, the additional cost is bounded linearly by a small factor, making our approach scale well with input length.

Pagination: In theory, an LLM could read a document and directly provide the pagination in a single pass, so the minimum number of words the LLM must process is the length of the document. Our pagination algorithm splits the document into chunks of at most `max_words`, and then guarantees

The following text is what you remember from reading an article and a multiple choice question related to it.
You may read 1 to 5 page(s) of the article again to refresh your memory to prepare yourself for the question.
Please respond with which page(s) you would like to read. For example, if you only need to read Page 8, respond with “I want to look up Page [8] to ...”; if you would like to read Page 7 and 12, respond with “I want to look up Page [7, 12] to ...”; if you would like to read Page 2, 3, 7, 15 and 18, respond with “I want to look up Page [2, 3, 7, 15, 18] to ...”.

DO NOT select more pages if you don’t need to.
You don’t need to answer the question yet.

Text:
{GIST MEMORY}
Question:
{QUESTION}

Figure 4: Example parallel lookup prompt (ReadAgent-P).

我们在提示中使用“缩短”一词来生成这些总结性要点，因为这通常有助于保持叙事流，使其在拼接时更自然。在我们的实验中，使用“总结”一词往往会产生结构重组的摘要。

原始页面大小是决定要点压缩程度的关键因素。假设我们考虑的最小文本单位是一个段落。直观上，一个段落很可能与其相邻段落存在一定量的互信息。

因此，我们将文本分组的块越大，能去除的重复信息就越多。经验表明，使用大语言模型压缩更大的文本块也倾向于移除更多细节，这可能影响性能。我们通过调整分页中的最小词数和最大词数来控制页面大小。这种权衡关系在附录A中进行了研究。

请缩短以下段落。只需给我一个缩短后的版本。不要解释原因。

段落:
{页面文本}

图3：要点提示示例。

3.2 并行与顺序交互式查找

对于涉及长文档的特定任务，我们希望ReadAgent除了利用其要点记忆外，还能采取行动在原始文本中查找相关细节。由于要点记忆已与页码关联，我们只需提示大语言模型根据具体任务回答它希望查找并重新阅读哪个（些）页面。接下来我们将讨论两种查找策略：一次性并行查找所有页面（

ReadAgent-P）以及按顺序一次查找一个页面（**ReadAgent-S**）。

ReadAgent-P 如图4中的问答示例所示，我们通常会设定一个最大可查找页面数，但同时指示它尽可能少地使用页面，以避免不必要的计算开销和干扰信息。

以下文本是您阅读一篇文章及相关多项选择题后所记住的内容。
您可以重新阅读该文章的1到5个页面来刷新记忆，为回答问题做准备。
请回复您希望阅读哪个（些）页面。例如，若只需阅读第8页，请回复“I want to look up Page [8] to ...”；若希望阅读第7页和第12页，请回复“I want to look up Page [7, 12] to ...”；若希望阅读第2、3、7、15和18页，请回复“I want to look up Page [2, 3, 7, 15, 18] to ...”。

除非必要，否则请勿选择更多页面。
你暂时还不需要回答问题。

文本:
{要点记忆}
问题:
{问题}

图4：并行查找提示示例（ReadAgent-P）。

以下提示展示了并行查找，其中模型响应单个提示请求多个页面。

选定的原始页面会替换记忆中相应位置的要点，从而保持整体叙事流。然后，我们再次向大语言模型提示任务和更新后的记忆，并要求它解决该任务。

ReadAgent-S 我们还研究了顺序查找策略（图5），即模型每次请求一个页面，直至达到某个最大页面数。在顺序查找中，模型在决定扩展哪个页面之前，能够看到先前已扩展的页面。这使得模型能比并行查找获取更多信息，因此我们可能预期它在某些情况下表现更好。然而，与模型交互次数的增加会提升计算成本，因此顺序查找应仅用于能带来明显收益的任务。

3.3 计算权衡与可扩展性

片段分页、记忆要点提取和交互式查找都需要迭代推理。如下文所示，其额外成本以一个较小的因子线性增长，这使得我们的方法能随着输入长度良好地扩展。

分页：理论上，大语言模型可以读取文档并直接在一次处理中提供分页，因此大语言模型必须处理的最小词数就是文档的长度。我们的分页算法将文档分割成最多包含最大词数的块，并保证

The following text is what you remember from reading a meeting transcript, followed by a question about the transcript. You may read multiple pages of the transcript again to refresh your memory and prepare to answer the question. Each page that you re-read can significantly improve your chance of answering the question correctly. Please specify a SINGLE page you would like to read again or say "STOP". To read a page again, respond with "Page \$PAGE_NUM", replacing \$PAGE_NUM with the target page number. You can only specify a SINGLE page in your response at this time. To stop, simply say "STOP". DO NOT answer the question in your response.

Text:
{GISTS WITH IN-LINE EXPANDED PAGES}
Pages re-read already (DO NOT ask to read them again):
{LIST OF PAGE NUMBERS ALREADY READ}
Question:
{QUESTION}
Specify a SINGLE page to read again, or say STOP:

Figure 5: Example sequential lookup prompt (ReadAgent-S).

that at least $\min_words$ are consumed at each step. Thus, the ratio $\frac{\max_words}{\min_words}$ gives an upper bound on how many times the word length of the document the LLM must process using our algorithm. **Gisting:** Memory gisting is one additional pass of the raw input words, since each page is gisted independently. **Look-ups:** Parallel look-ups are conditioned on gists instead of the full text, and thus will be much shorter than one pass of the raw input words. Each step of a sequential look-up is similar to parallel look-ups and the overall cost is capped with the maximum number of look-ups allowed. **Response:** Finally, answering is also similar to parallel look-ups. There is additional overhead from the prompt templates, of course.

On the other hand, as generating gists is an one-time effort while the look-up and response steps operate mostly on gists that are much shorter than the original text, the one-time effort can be amortized when the same context is reused for multiple tasks. Thus, in such settings, ReadAgent can reduce the overall number of tokens to process. In particular, directly answering from the original QuALITY dev set (230 articles and 2086 questions) is 8,708,434 words consumed by the LLM, whereas using ReadAgent with 1-page lookup is 6,499,856 words (25.4% saving) and up-to-5-page lookup is 7,503,084 words (13.8% saving). We can expect the savings to be more significant with higher compression rate and more downstream tasks.

3.4 READAGENT VARIANTS

In Appendix F, we discuss variants of ReadAgent that can be useful in different problem settings, including when the target task is known prior to reading the long document. In Appendix D, we describe adapting ReadAgent to work in the web navigation setting.

4 EXPERIMENTS

We evaluate ReadAgent’s long-document reading comprehension ability on three long-context question-answering challenges: QuALITY Pang et al. (2022), NarrativeQA Kočiský et al. (2018) and QMSum Zhong et al. (2021). Although ReadAgent does not require any model training, we develop the proposed method on the training sets and test on the validation, test and/or development sets to avoid any risk of overfitting system hyperparameters.

In this work, we primarily use the instruction-tuned PaLM 2-L Anil et al. (2023) for our experiments and evaluation. The context length of PaLM 2-L is 8K tokens. Details of the model can be found in Anil et al. (2023). Additionally, we provide GPT-3.5² results in Appendix B, and experimental results on the web navigation setting in Appendix D.

One important performance measure of the techniques considered here is the **compression rate (CR)**. As we want to measure the longest LLM context length that ReadAgent requires versus full-context length, we define this as $CR \equiv 100 * (1 - \frac{\text{word-count}(\text{in-context text})}{\text{word-count}(\text{full-context text})})$ at the final response query, where the in-context text (gists and retrieved pages) length is the longest among all inference steps.

²<http://openai.com/api/>

以下文本是您阅读会议记录后所记住的内容，其后附有一个关于该记录的问题。
您可以重新阅读记录的多个页面来刷新记忆，并准备回答问题。您重新阅读的每个页面都能显著提高您正确回答问题的几率。请指定您想重新阅读的单个页面，或说“STOP”。若要重新阅读某个页面，请回复“Page \$PAGE_NUM”，并将\$PAGE_NUM替换为目标页码。此时你只能在回复中指定一个单独的页面。若要停止，只需说“STOP”。不要在回复中回答问题。

文本：
{包含内联扩展页面的要点}
已重新阅读的页面（请勿要求再次阅读）：
{已阅读的页码列表}
问题：
{问题}
指定一个要重新阅读的单一页面，或说停止：

图5：顺序查找提示示例（ReadAgent-S）。

每一步至少消耗最小词数。因此， $\frac{\text{最大词数}}{\text{最小词数}}$ 的比率给出了使用我们算法时，大语言模型必须处理的文档词长大小的上限。 **要点提取：**记忆要点提取是对原始输入词进行的额外一次处理，因为每个页面都是独立提取要点的。 **查找：**并行查找是基于要点而非全文进行的，因此将比一次原始输入词的处理短得多。顺序查找的每一步与并行查找类似，其总成本受允许的最大查找次数限制。 **响应：**最后，回答也与并行查找类似。当然，提示模板会带来额外的开销。

另一方面，由于生成要点是一次性工作，而查找和回答步骤主要在比原始文本短得多的要点上进行，当同一上下文被重复用于多个任务时，这次性工作的成本可以被分摊。因此，在此类场景中，ReadAgent可以减少需要处理的词元总数。具体而言，直接基于原始QuALITY开发集（230篇文章和2086个问题）进行回答会消耗大语言模型8,708,434个词，而使用ReadAgent进行1页查找则消耗6,499,856个词（节省25.4%），进行最多5页查找则消耗7,503,084个词（节省13.8%）。我们可以预期，在压缩率更高、下游任务更多的情况下，节省的效果会更加显著。

3.4 READAGENT 变体

在附录 F 中，我们讨论了 ReadAgent 的几种变体，它们在不同的问题场景中可能很有用，包括在阅读长文档之前已知目标任务的情况。在附录 D 中，我们描述了如何使 ReadAgent 适应网络导航场景。

4 实验

我们在三个长上下文问答挑战上评估 ReadAgent 的长文档阅读理解能力：QuALITY（Pang 等人，2022）、NarrativeQA（Kočiský 等人，2018）和 QMSum（Zhong 等人，2021）。尽管 ReadAgent 不需要任何模型训练，但我们在训练集上开发所提出的方法，并在验证集、测试集和/或开发集上进行测试，以避免系统超参数过拟合的任何风险。在本工作中，我们主要使用指令调优的 PaLM 2-L（Anil 等人，2023）进行实验和评估。PaLM 2-L 的上下文长度为 8K 词元。该模型的详细信息可在 Anil 等人（2023）中找到。此外，我们在附录 B 中提供了 GPT-3.5² 的结果，并在附录 D 中提供了网络导航场景下的实验结果。

此处所考量技术的一项重要性能指标是**压缩率（CR）**。由于我们希望衡量 ReadAgent 所需的最长 LLM 上下文长度与完整上下文长度之比，我们将其定义为在最终响应查询时的 $CR \equiv 100 * (1 - \frac{\text{词数}(\text{上下文内文本})}{\text{词数}(\text{完整上下文文本})})$ ，其中上下文内文本（要点和检索到的页面）的长度是所有推理步骤中最长的。

²<http://openai.com/api/>

4.1 LLM RATERS

NarrativeQA and QMSum both have one or more free-form reference responses. They are typically evaluated using syntactic matching metrics such as ROUGE (Lin, 2004) F-Measure. We additionally evaluate these datasets using an automatic LLM Rater as an alternative to human evaluation similar to Peng et al. (2023); Chiang et al. (2023); Zheng et al. (2023); Chiang & Lee (2023).

In our implementation, we prompt the LLM to look at the question or instruction and compare the model’s answer to the reference answer. The “Strict LLM Rater Prompt” (Figure 6) is for judging whether there is an exact match, and the “Permissive LLM Rater Prompt” (Figure 7) is for judging whether there is an exact match or a partial match. We apply both prompts to all model responses. If either rater decides there is an exact match, we count it as an exact match. If the strict rater is negative but the permissive rater detects a partial match, we count it as a partial match. Otherwise, it’s not a match. In the case that there are multiple reference answers, the response is compared against each reference answer in turn, and the highest rating is returned.

Based on these raters, we define two different scores: **LLM-Rating-1 (LR-1)** is a strict evaluation score, where we count the percentage of exact matches over all examples; **LLM-Rating-2 (LR-2)** is permissive, where we count the percentage of exact and partial matches.

4.2 BASELINE METHODS

Retrieval-Augmented Generation (RAG) As discussed in Section 2, RAG Lewis et al. (2020) is a popular approach to extend access to a large amount of text beyond what can fit in the LLM context window. In this paper we compare ReadAgent to RAG baselines using conventional retrieval methods to find relevant “pages” in a long text, where we reuse the pages generated by ReadAgent. We consider two relevance methods: Okapi BM25 Robertson et al. (2009) and neural retrieval based on the Gemini API embedding model (models/embedding-001)³. The neural retrieval relevance score is defined as the dot product between the question embedding vector and each page (or gist memory embedding vector in the case of NarrativeQA, see Section 4.4). For reading comprehension tasks, the pages are ranked by relevance to each question, and we prompt the LLM to look at the top- k pages as context for answering the question. In most retrieval settings, the database of documents is quite large, which makes the retrieval task more challenging. In our setting, ReadAgent and retrieval methods all use a per-document database, rather than per-dataset. For example, in QuALITY, there are hundreds of articles, each with multiple questions. The database for retrieval in each question is only the extracted pages from the corresponding article (typically less than 20 pages), rather than the thousands of pages from the entire dataset.

Full or Truncated Text Content The maximum length of QuALITY dev articles is $\sim 6,000$ words, which can fit into the PaLM 2-L context window. This allows us to evaluate ReadAgent against directly using the full long document for long-context reading comprehension. The maximum length of QMSum is over 26,000 words. Consequently, we choose to truncate the text to close to the context window limit (6,000 words for PaLM 2-L experiments) to ensure that the truncated text fits in the LLM’s context, though this would generally be a weaker baseline. Finally, since the average length of NarrativeQA documents significantly exceeds the context window, it is less meaningful to perform the truncated-context comparison.

³<https://ai.google.dev/models/gemini>

After reading some text, John was given the following question about the text:
{QUESTION TEXT}
John’s answer to the question was:
{MODEL RESPONSE TEXT}
The ground truth answer was:
{REFERENCE RESPONSE TEXT}
Does John’s answer agree with the ground truth answer? Please answer YES or NO.

Figure 6: Strict LLM Rater prompt.

After reading some text, John was given the following question about the text:
{QUESTION TEXT}
John’s answer to the question was:
{MODEL RESPONSE TEXT}
The ground truth answer was:
{REFERENCE RESPONSE TEXT}
Does John’s answer agree with the ground truth answer? Please answer “Yes”, “Yes, partially”, or “No”. If John’s response has any overlap with the ground truth answer, answer “Yes, partially”. If John’s response contains the ground truth answer, answer “Yes”. If John’s response is more specific than the ground truth answer, answer “Yes”.

Figure 7: Permissive LLM Rater prompt.

4.1 LLM 评分器

NarrativeQA 和 QMSum 都包含一个或多个自由形式参考回答。它们通常使用 ROUGE (Lin, 2004) F-Measure 等句法匹配指标进行评估。我们还使用自动 LLM 评分器作为人工评估的替代方案来评估这些数据，类似于 Peng 等人 (2023); Chiang 等人 (2023); Zheng 等人 (2023); Chiang 和 Lee (2023) 的做法。

在我们的实现中，我们提示大语言模型查看问题或指令，并将模型的答案与参考答案进行比较。

“严格 LLM 评分提示” (图6) 用于判断是否存在完全匹配，而“宽松 LLM 评分提示” (图7) 用于判断是否存在完全匹配或部分匹配。我们将这两种提示应用于所有模型响应。如果任一评分器判定存在完全匹配，我们将其计为完全匹配。如果严格评分器给出否定结果，但宽松评分器检测到部分匹配，我们将其计为部分匹配。否则，即为不匹配。在存在多个参考答案的情况下，响应会依次与每个参考答案进行比较，并返回最高评分。

基于这些评分器，我们定义了两个不同的分数：**LLM 评分-1 (LR-1)** 是一个严格的评估分数，我们计算所有示例中精确匹配的百分比；**LLM 评分-2 (LR-2)** 是宽松的，我们计算精确匹配和部分匹配的百分比。

4.2 基线方法

检索增强生成 (RAG) 如第2节所述，RAG (Lewis 等人, 2020) 是一种流行的方法，用于扩展对大语言模型上下文窗口容纳范围之外的大量文本的访问。在本文中，我们将 ReadAgent 与使用传统检索方法在长文本中查找相关“页面”的 RAG 基线进行比较，其中我们复用了 ReadAgent 生成的页面。我们考虑两种相关性方法：Okapi BM25 (Robertson 等人, 2009) 和基于 Gemini API 嵌入模型 (models/embedding-001) 的神经

检索³。神经检索的相关性分数定义为问题嵌入向量与每个页面（或在 NarrativeQA 情况下为要点记忆嵌入向量，见第4.4节）之间的点积。对于阅读理解任务，页面根据与每个问题的相关性进行排序，我们提示大语言模型查看最相关的 k 个页面作为回答问题的上下文。在大多数检索设置中，文档数据库相当大，这使得检索任务更具挑战性。在我们的设置中，ReadAgent 和检索方法都使用按文档的数据库，而不是按数据集的。例如，在 QuALITY 中，有数百篇文章，每篇都有多个问题。每个问题的检索数据库仅包含从相应文章中提取的页面（通常少于20页），而不是来自整个数据集的数千页。

完整或截断的文本内容 QuALITY 开发集文章的最大长度为 $\sim 6,000$ 词，这可以放入 PaLM 2-L 的上下文窗口中。这使得我们能够将 ReadAgent 与直接使用完整长文档进行长上下文阅读理解的方法进行比较评估。QMSum 的最大长度超过 26,000 词。因此，我们选择将文本截断至接近上下文窗口限制（PaLM 2-L 实验为 6,000 词），以确保截断后的文本能放入大语言模型的上下文中，尽管这通常是一个较弱的基线。最后，由于 NarrativeQA 文档的平均长度显著超过上下文窗口，进行截断上下文比较的意义不大。

³<https://ai.google.dev/models/gemini>

阅读完一些文本后，John 被问到以下关于该文本的问题：
{问题文本}
John 对该问题的回答是：
{模型响应文本}
地面真值答案是：
{参考响应文本}
John 的答案是否与地面真值答案一致？请回答是或否。

图6：严格 LLM 评分提示。

阅读完一些文本后，John 被问到以下关于该文本的问题：{问题文本} John 对该问题的回答是：{模型响应文本} 地面真值答案是：{参考响应文本} John 的答案是否与地面真值答案一致？请回答“是”、“是，部分一致”或“否”。如果 John 的响应与地面真值答案有任何重叠，请回答“是，部分一致”。如果 John 的响应包含了地面真值答案，请回答“是”。如果 John 的响应比地面真值答案更具体，请回答“是”。

图 7：宽松 LLM 评分提示。

Gist Memory We can also attempt to solve the given task by reasoning directly over the gist memory. Doing so helps us understand not only the importance of interactive look-up but also how using the LLM-compressed information alone compares to the full content and retrieval baselines.

4.3 QUALITY

QuALITY Pang et al. (2022) is a four-way multiple choice question answering challenge with text data from several different sources. QuALITY is evaluated using accuracy, with 25% corresponding to chance performance.

The dev set has an average length of 4,122 words and a maximum of 5,967. The gist memory has an average length of 650 words and a maximum of 1,264. Figure 8 shows the word statistics for the original text and the gists. The compression rate of the gists is 85.53%. See Appendix H for QuALITY pagination hyperparameters.

Table 1 shows the experimental results on QuALITY. The performance of ReadAgent increases as we increase the maximum number of pages allowed for look-up. ReadAgent-P (Look up 1-6 pages) achieves 86.91% and ReadAgent-S (Look up 1-6 pages) achieves 87.17% in accuracy. Notably, starting from ReadAgent (Look up 1-2 pages), it outperforms all baselines methods including using the full original text, which could have been an upper bound on the performance – every other method reduces the amount of text the LLM considers before generating its response. However, this is not a surprising result. Prior work shows that current LLMs are not able to effectively use the full long context window (Liu et al., 2023), potentially due to training data sparsity, and distracting information can also reduce performance (Shi et al., 2023; Weston & Sukhbaatar, 2023). The corresponding compression rate of ReadAgent (Look up 1-2 pages) is 72.17%, meaning that $\sim 3.5\times$ as many tokens can fit in the window after gisting.

4.4 NARRATIVEQA

NarrativeQA (Kočíský et al., 2018) has the longest context length on average among the three reading comprehension datasets we choose. The dataset is divided into books (Gutenberg) and movie scripts. The Gutenberg test set have 70,619 words on average, and the maximum is 343,910 words; the movie scripts test set have 29,963 on average, and the maximum is 63,957 words. As the reference answers are free-form, we evaluate based on ROUGE (Lin, 2004) and the LLM Ratings (Section 4.1). We replace the original main texts with the HTML-stripped text from SCROLLS (Shaham et al., 2022).

Because of the length of NarrativeQA articles, to fit the gists into the context window, we significantly expand the page size, resulting in stronger compression (Section 3.1). For example, the Gutenberg gists from the test set have 2,217 words on average and the maximum is 6,471 words, whereas the movie script gists have 2,155 words on average and the maximum is 4,511 words. Figures 10 and 11 (appendix) show the word statistics for the original text and the gists in Gutenberg and movie scripts respectively. The compression rate of the gists is 96.80% for Gutenberg texts and 91.98% for movie scripts. See Appendices H and I for NarrativeQA pagination hyperparameters and more details.

For the neural retrieval models, we use the gist memory embedding vectors rather than the page embedding vectors because the Gemini API embedding model is limited to 10,000 characters (or less than 2,000 tokens, in expectation), which is too short for embedding full pages in our NarrativeQA experiments. However, using those embedding vectors, we then return the original pages to the LLM

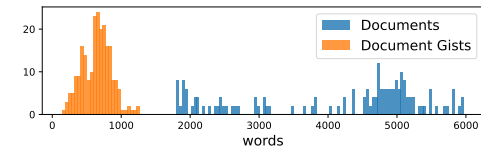


Figure 8: Histogram of QuALITY document and gist word counts.

Method	CR (# LU)	Accuracy
BM25 Retrieval		
Top-1	89.27% (1)	70.34% $\pm$ 0.06
Top-2	78.96% (2)	79.05% $\pm$ 0.05
Top-3	68.50% (3)	82.65% $\pm$ 0.05
Top-4	58.57% (4)	84.42% $\pm$ 0.13
Neural Retrieval with Gemini API		
Top-1	89.91% (1)	71.32% $\pm$ 0.19
Top-2	80.08% (2)	79.02% $\pm$ 0.10
Top-3	70.28% (3)	83.41% $\pm$ 0.10
Top-4	60.68% (4)	84.88% $\pm$ 0.03
Full Raw Content	0%	85.83% $\pm$ 0.19
GistMem	85.53%	77.52% $\pm$ 0.13
ReadAgent-P		
Look up 1 pg	76.00% (1.0)	84.13% $\pm$ 0.10
Look up 1-2 pgs	72.17% (1.6)	86.16% $\pm$ 0.12
Look up 1-3 pgs	69.36% (2.0)	86.59% $\pm$ 0.10
Look up 1-4 pgs	67.73% (2.2)	86.86% $\pm$ 0.00
Look up 1-5 pgs	66.45% (2.3)	86.83% $\pm$ 0.10
Look up 1-6 pgs	64.75% (2.5)	86.91% $\pm$ 0.08
ReadAgent-S 1-6 pgs	58.53% (3.2)	87.17% $\pm$ 0.18

Table 1: QuALITY results on the dev set of 230 docs and 2086 questions using PaLM 2-L. **CR** is the compression rate. **# LU** is the number of lookups. We report means and standard deviations across 3 runs, except where inconsequential (CR and # LU).

要点记忆我们也可以尝试直接基于要点记忆进行推理来解决给定任务。这样做不仅有助于我们理解交互式查找的重要性，还能了解仅使用大语言模型压缩的信息与完整内容及检索基线相比效果如何。

4.3 QUALITY

QuALITY Pang 等人 (2022) 是一个四选一的多选题问答挑战，其文本数据来自多个不同来源。QuALITY 使用准确率进行评估，25% 对应随机猜测的表现。

开发集的平均长度为4,122个单词，最大长度为5,967个单词。要点记忆的平均长度为650个单词，最大长度为1,264个单词。图8显示了原始文本和要点的单词统计信息。要点的压缩率为85.53%。QuALITY分页超参数请参见附录H。

表1展示了在QuAL-ITY上的实验结果。ReadAgent的性能随着我们允许查找的最大页数增加而提升。ReadAgent-P（查找1-6页）达到了86.91%的准确率，而ReadAgent-S（查找1-6页）达到了87.17%的准确率。值得注意的是，从ReadAgent（查找1-2页）开始，其表现就超过了所有基线方法，包括使用完整原始文本的方法——后者本可能是性能的上限，因为其他所有方法都在大语言模型生成回答前减少了其需考虑的文本量。然而，这并非一个令人惊讶的结果。先前的研究表明，当前的大语言模型无法有效利用完整的超长上下文窗口（Liu等人，2023）。

这可能是由于训练数据稀疏性所致，而干扰信息也会降低性能（Shi等人，2023；Weston & Sukhbaatar, 2023）。ReadAgent（查阅1-2页）对应的压缩率为72.17%，这意味着 $\sim 3.5\times$ 经过要点提取后，窗口可容纳的词元数量大幅增加。

4.4 NARRATIVEQA

NarrativeQA (Kočíský等人，2018) 在我们选取的三个阅读理解数据集中平均上下文长度最长。该数据集分为书籍（古登堡计划）和电影剧本两部分。古登堡测试集平均有70,619个单词，最长达343,910个单词；电影剧本测试集平均有29,963个单词，最长达63,957个单词。由于参考答案为自由形式，我们基于ROUGE (Lin, 2004) 和LLM评分（第4.1节）进行评估。我们将原始正文替换为来自SCROLLS (Shaham等人，2022) 的去掉HTML的文本。由于NarrativeQA文章篇幅较长，为了将要点纳入上下文窗口，我们大幅扩展了页面大小，从而实现了更强的压缩（第3.1节）。例如，测试集中的古腾堡计划要点平均有2,217个单词，最长达6,471个单词；而电影剧本要点平均有2,155个单词，最多为4,511个单词。图10和图11（附录）分别展示了古腾堡计划文本和电影剧本的原始文本与要点的字数统计。要点的压缩率在古腾堡文本中为96.80%，在电影剧本中为91.98%。有关NarrativeQA分页超参数及更多细节，请参见附录H和I。

对于神经检索模型，我们使用要点记忆嵌入向量而非页面嵌入向量，因为Gemini API嵌入模型限制在10,000个字符（或预期少于2,000个词元），这对于我们在NarrativeQA实验中嵌入完整页面而言太短了。然而，使用这些嵌入向量后，我们会将原始页面返回给大语言模型。

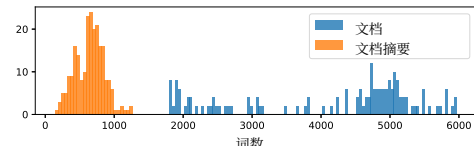


图8: QuALITY 文档与要点词数分布直方图。

方法	压缩率 (# LU)	准确率
BM25 检索		
Top-1	89.27% (1)	70.34% $\pm$ 0.06
前两名	78.96% (2)	79.05% $\pm$ 0.05
Top-3	68.50% (3)	82.65% $\pm$ 0.05
前四	58.57% (4)	84.42% $\pm$ 0.13
使用 Gemini API 的神经检索		
Top-1	89.91% (1)	71.32% $\pm$ 0.19
前两名	80.08% (2)	79.02% $\pm$ 0.10
Top-3	70.28% (3)	83.41% $\pm$ 0.10
前四	60.68% (4)	84.88% $\pm$ 0.03
完整原始内容	0%	85.83% $\pm$ 0.19
GistMem	85.53%	77.52% $\pm$ 0.13
ReadAgent-P		
查找1页	76.00% (1.0)	84.13% $\pm$ 0.10
查找1-2页	72.17% (1.6)	86.16% $\pm$ 0.12
查找1-3页	69.36% (2.0)	86.59% $\pm$ 0.10
查找1-4页	67.73% (2.2)	86.86% $\pm$ 0.00
查阅 1-5 页	66.45% (2.3)	86.83% $\pm$ 0.10
查阅 1-6 页	64.75% (2.5)	86.91% $\pm$ 0.08
ReadAgent-S 1-6 页	58.53% (3.2)	87.17% $\pm$ 0.18

表1: 使用PaLM 2-L在包含230篇文档和2086个问题的开发集上的QuALITY结果。**压缩率**是压缩率。**# 查找次数**是查找次数。我们报告了3次运行的平均值和标准差，除非无关紧要（压缩率和# 查找次数）。

	Gutenberg Validation (58 docs & 1743 questions)						Gutenberg Test (177 docs & 5207 questions)					
Method	CR (# LU)	LR-1	LR-2	R-1	R-2	R-L	CR (# LU)	LR-1	LR-2	R-1	R-2	R-L
BM25 Retrieval												
Top-1	97.63% (1)	39.01%	50.14%	0.166	0.061	0.156	97.42% (1)	43.5%	55.33%	0.176	0.065	0.165
Top-2	95.24% (2)	49.34%	60.76%	0.203	0.079	0.191	94.80% (2)	51.70%	64.53%	0.206	0.082	0.194
Top-3	93.34% (3)	52.73%	63.68%	0.208	0.080	0.195	93.02% (3)	52.97%	66.03%	0.210	0.083	0.197
Top-4	92.47% (4)	53.59%	64.26%	0.211	0.082	0.197	92.27% (4)	53.60%	66.16%	0.210	0.084	0.195
Neural Retrieval with Gemini API												
Top-1	98.19% (1)	34.25%	46.53%	0.146	0.051	0.134	98.14% (1)	36.47%	47.8%	0.150	0.054	0.140
Top-2	96.30% (2)	44.69%	54.96%	0.180	0.069	0.167	96.15% (2)	44.48%	56.17%	0.182	0.070	0.170
Top-3	94.62% (3)	46.24%	57.31%	0.191	0.077	0.178	94.42% (3)	48.97%	60.73%	0.195	0.076	0.183
Top-4	93.45% (4)	48.59%	59.21%	0.196	0.079	0.184	93.25% (4)	50.62%	62.05%	0.203	0.080	0.191
GistMem	96.89%	55.31%	68.22%	0.233	0.091	0.218	96.80%	55.79%	71.19%	0.231	0.092	0.217
ReadAgent-P												
Look up 1 pg	95.15% (0.94)	58.92%	71.89%	0.244	0.101	0.230	94.84% (0.93)	59.98%	73.23%	0.240	0.098	0.226
Look up 1-2 pgs	94.79% (1.23)	59.84%	72.29%	0.239	0.098	0.224	94.36% (1.34)	59.19%	72.65%	0.231	0.091	0.218
Look up 1-3 pgs	94.39% (1.50)	59.84%	71.89%	0.240	0.098	0.226	94.03% (1.61)	59.63%	72.84%	0.230	0.093	0.217
ReadAgent-S 1-2 pgs	94.35% (1.38)	57.89%	71.14%	0.239	0.097	0.225	93.86% (1.46)	60.48%	72.48%	0.232	0.095	0.219
ReadAgent-S 1-3 pgs	94.08% (1.57)	58.52%	71.49%	0.242	0.098	0.229	93.67% (1.57)	60.55%	72.79%	0.231	0.095	0.219
Movie Validation (57 docs & 1699 questions)												
Movie Test (172 docs & 5139 questions)												
BM25 Retrieval												
Top-1	97.07% (1)	32.67%	42.61%	0.156	0.058	0.144	96.61% (1)	33.64%	43.34%	0.154	0.054	0.143
Top-2	94.12% (2)	39.97%	50.21%	0.187	0.070	0.174	93.81% (2)	42.50%	53.05%	0.191	0.072	0.178
Top-3	91.18% (3)	43.61%	53.91%	0.198	0.077	0.185	91.00% (3)	46.97%	57.52%	0.207	0.080	0.193
Top-4	88.24% (4)	46.85%	57.62%	0.210	0.084	0.198	88.19% (4)	50.18%	60.13%	0.217	0.085	0.202
Neural Retrieval with Gemini API												
Top-1	97.07% (1)	32.02%	41.44%	0.153	0.053	0.142	96.67% (1)	37.24%	46.22%	0.130	0.043	0.118
Top-2	94.19% (2)	43.20%	51.38%	0.160	0.057	0.148	93.90% (2)	46.49%	54.60%	0.164	0.061	0.151
Top-3	91.29% (3)	47.56%	56.21%	0.176	0.064	0.163	91.14% (3)	50.69%	58.92%	0.186	0.071	0.172
Top-4	88.38% (4)	49.09%	59.33%	0.193	0.075	0.180	88.36% (4)	52.13%	59.41%	0.184	0.072	0.171
GistMem	92.09%	52.56%	64.39%	0.242	0.103	0.227	91.98%	54.68%	64.00%	0.248	0.105	0.234
ReadAgent-P												
Look up 1 pg	89.20% (0.99)	53.38%	65.57%	0.247	0.106	0.233	89.22% (0.98)	57.68%	68.01%	0.274	0.116	0.260
Look up 1-2 pgs	87.68% (1.52)	54.62%	65.63%	0.238	0.098	0.223	88.10% (1.39)	58.24%	68.81%	0.270	0.115	0.255
Look up 1-3 pgs	86.57% (1.91)	54.91%	65.86%	0.241	0.099	0.225	86.73% (1.89)	58.82%	69.12%	0.272	0.116	0.257
ReadAgent-S 1-2 pgs	86.36% (1.98)	59.33%	68.28%	0.203	0.082	0.188	85.92% (1.98)	63.33%	72.06%	0.214	0.086	0.199
ReadAgent-S 1-3 pgs	83.56% (2.95)	59.45%	68.81%	0.210	0.087	0.195	83.18% (2.95)	64.53%	73.06%	0.217	0.090	0.202

Table 2: NarrativeQA results (PaLM 2-L). **R-1**, **R-2**, and **R-L** are ROUGE F-Measures. **LR-1**, and **LR-2** are LLM-Ratings.

context as normal, and use those pages as described in Section 4.2. Because the Gutenberg texts and the movie scripts have significantly different distributions, we present the results separately in Table 2. ReadAgent again outperforms all the baselines across all subsets of NarrativeQA.

4.5 QMSum

QMSum (Zhong et al., 2021) consists of meeting transcripts on various topics and associated questions or instructions. We use the concatenated version of QMSum provided by SCROLLS (Shaham et al., 2022). The transcripts tend to be quite long, ranging in length from 1,000 to 26,300 words, with an average length of about 10,000 words. Figure 13 shows the histograms of word counts for the QMSum training set. The answers are free form text, so the standard evaluation metric is ROUGE F-Measure. We additionally evaluate using our LLM Ratings (Section 4.1). See Appendices H and J for hyperparameters and additional results.

In Tables 3 and 11, we see that performance improves as the compression rate decreases, so techniques that look up more pages tend to do better than techniques that look up fewer pages. We also see that ReadAgent-S substantially outperforms ReadAgent-P (and all baselines). This performance improvement comes at a cost of up to six times as many requests in the retrieval phase. Since other datasets don’t have such a strong performance improvement, we suspect that QMSum is in some sense a more challenging dataset, requiring the model to actively search through the gisted transcript to locate relevant information. This hypothesis seems reasonable, as meeting transcripts are much less structured than the documents, books, and movies found in QuALITY and NarrativeQA.

A large fraction of the tasks in QMSum are a request to provide a summary, rather than a concrete question about some content in the meeting. For many of these, the LLM refuses to look up any pages, instead responding with “I don’t need to look up any pages. I can summarize the whole meeting based on what I already remember.”, for example. Consequently, the average number of pages looked up for ReadAgent is much lower than the maximum allowed. However, on the tasks that actually involve a question, ReadAgent tends to use most or all of the available lookup pages.

In Tables 3 and 11, the ROUGE scores by themselves don’t always show a clear trend. This is because as the length of the texts increase (corresponding to the compression rates decreasing), the response lengths increase as well. Longer response lengths result in lower ROUGE precision

	古腾堡验证 (58 篇文档 & 1743 个问题)						古腾堡测试 (177 篇文档 & 5207 个问题)					
方法	压缩率 (F1 L1/L)	LR-1	LR-2	R-1	R-2	R-L	压缩率 (F1 L1/L)	LR-1	LR-2	R-1	R-2	R-L
BM25 检索												
Top-1	97.63% (1)	39.01%	50.14%	0.166	0.061	0.156	97.42% (1)	43.5%	55.33%	0.176	0.065	0.165
前两名	95.24% (2)	49.34%	60.76%	0.203	0.079	0.191	94.80% (2)	51.70%	64.53%	0.206	0.082	0.194
Top-3	93.34% (3)	52.73%	63.68%	0.208	0.080	0.195	93.02% (3)	52.97%	66.03%	0.210	0.083	0.197
前四	92.47% (4)	53.59%	64.26%	0.211	0.082	0.197	92.27% (4)	53.60%	66.16%	0.210	0.084	0.197
使用 Gemini API 进行神经检索												
Top-1	98.19% (1)	34.25%	46.53%	0.146	0.051	0.134	98.14% (1)	36.47%	47.8%	0.150	0.054	0.140
前两名	96.30% (2)	44.69%	54.96%	0.180	0.069	0.167	96.15% (2)	44.48%	56.17%	0.182	0.070	0.170
Top-3	94.62% (3)	46.24%	57.31%	0.191	0.077	0.178	94.42% (3)	48.97%	60.73%	0.195	0.076	0.183
前四	93.45% (4)	48.59%	59.21%	0.196	0.079	0.184	93.25% (4)	50.62%	62.05%	0.203	0.080	0.191
GistMem	96.89%	55.31%	68.22%	0.233	0.091	0.218	96.80%	55.79%	71.19%	0.231	0.092	0.217
ReadAgent-P												
查找1页	95.15% (0.94)	58.92%	71.89%	0.244	0.101	0.230	94.84% (0.93)	59.98%	73.23%	0.240	0.098	0.226
查找1-2页	94.79% (1.23)	59.84%	72.29%	0.239	0.098	0.224	94.36% (1.34)	59.19%	72.65%	0.231	0.091	0.218
查找1-3页	94.39% (1.50)	59.84%	71.89%	0.240	0.098	0.226	94.03% (1.61)	59.63%	72.84%	0.230	0.093	0.217
ReadAgent-SI-2 页	94.35% (1.38)	57.89%	71.14%	0.239	0.097	0.225	93.86% (1.46)	60.48%	72.48%	0.232	0.095	0.219
ReadAgent-SI-3 页	94.08% (1.57)	58.52%	71.49%	0.242	0.098	0.229	93.67% (1.57)	60.55%	72.79%	0.231	0.095	0.219
电影验证集 (57 个文档 & 1699 个问题)												
电影测试集 (172 个文档 & 5139 个问题)												
BM25 检索												
Top-1	97.07% (1)	32.67%	42.61%	0.156	0.058	0.144	96.61% (1)	33.64%	43.34%	0.154	0.054	0.143
前两名	94.12% (2)	39.97%	50.21%	0.187	0.070	0.174	93.81% (2)	42.50%	53.05%	0.191	0.072	0.178
Top-3	91.18% (3)	43.61%	53.91%	0.198	0.077	0.185	91.00% (3)	46.97%	57.52%	0.207	0.080	0.193
前四	88.24% (4)	46.85%	57.62%	0.210	0.084	0.198	88.19% (4)	50.18%	60.13%	0.217	0.085	0.202
使用 Gemini API 进行神经检索												
Top-1	97.07% (1)	32.02%	41.44%	0.153	0.053	0.142	96.67% (1)	37.24%	46.22%	0.130	0.043	0.118
前两名	94.19% (2)	43.20%	51.38%	0.160	0.057	0.148	93.90% (2)	46.49%	54.60%	0.164	0.061	0.151
Top-3	91.29% (3)	47.56%	56.21%	0.176	0.064	0.163	91.14% (3)	50.69%	58.92%	0.186	0.071	0.172
前四	88.38% (4)	49.09%	59.33%	0.193	0.075	0.180	88.36% (4)	52.13%	59.41%	0.184	0.072	0.171
GistMem	92.09%	52.56%	64.39%	0.242	0.103	0.227	91.98%	54.68%	64.00%	0.248	0.105	0.234
ReadAgent-P												
查找1页	89.20% (0.99)	53.38%	65.57%	0.247	0.106	0.233	89.22% (0.98)	57.68%	68.01%	0.274	0.116	0.260
查找1-2页	87.68% (1.52)	54.62%	65.63%	0.238	0.098	0.223	88.10% (1.39)	58.24%	68.81%	0.270	0.115	0.255
查找1-3页	86.57% (1.91)	54.91%	65.86%	0.241	0.099	0.225	86.73% (1.89)	58.82%	69.12%	0.272	0.116	0.257
ReadAgent-SI-2 页	86.36% (1.98)	59.33%	68.28%	0.203	0.082	0.188	85.92% (1.98)	63.33%	72.06%	0.214	0.086	0.199
ReadAgent-SI-3 pgs	83.56% (2.95)	59.45%	68.81%	0.210	0.087	0.195	83.18% (2.95)	64.53%	73.06%	0.217	0.090	0.202

表2: NarrativeQA结果 (PaLM 2-L) 。**R-1**、**R-2**和**R-L** 是ROUGE F值。**LR-1**和**LR-2**是LLM评分。

作为常规上下文，并按照第4.2节所述使用这些页面。由于古腾堡文本和电影剧本的分布存在显著差异，我们在表2中分别呈现结果。ReadAgent在所有NarrativeQA子集上再次超越了所有基线。

4.5 QMSum

QMSum (Zhong等人，2021) 包含各种主题的会议记录以及相关的问题或指令。我们使用由SCROLLS (Shaham等人，2022) 提供的QMSum拼接版本。这些记录往往非常长，长度范围从1,000到26,300词不等，平均长度约为10,000词。图13展示了QMSum训练集的词数直方图。答案采用自由文本形式，因此标准评估指标是ROUGE F值。我们还使用我们的LLM评分（第4节）进行了额外评估。有关超参数和更多结果，请参见附录H和J。

在表3和表11中，我们看到性能随着压缩率的降低而提升，因此查阅更多页面的技术往往比查阅较少页面的技术表现更好。我们还看到，ReadAgent-S 显著优于 ReadAgent-P（以及所有基线）。这种性能提升的代价是检索阶段的请求量最多增加了六倍。由于其他数据集没有表现出如此显著的性能提升，我们推测 QMSum 在某种意义上是一个更具挑战性的数据集，它要求模型主动在摘要转录本中搜索以定位相关信息。这个假设似乎是合理的，因为会议记录的结构性远不如 QuALITY 和 NarrativeQA 中的文档、书籍和电影。

QMSum 中很大一部分任务是要求提供摘要，而非针对会议中某些内容提出具体问题。对于其中许多任务，大语言模型会拒绝查阅任何页面，而是回应例如“我不需要查阅任何页面。我可以根据已有的记忆来总结整个会议。”因此，ReadAgent 查阅的平均页面数远低于允许的最大值。然而，在那些实际涉及问题的任务上，ReadAgent 倾向于使用大部分或全部可用的查阅页面。

在表3和表11中，ROUGE分数本身并不总是显示出清晰的趋势。这是因为随着文本长度的增加（对应压缩率的降低），响应长度也会增加。更长的响应长度会导致更低的ROUGE精确率，

Method	CR (# LU)	LLM Rating-1	LLM Rating-2	ROUGE-1	ROUGE-2	ROUGE-L	Resp. Length
BM25 Retrieval							
Top-1	95.69% (1.00)	32.48% ± 1.65	63.85% ± 1.51	27.53 ± 0.23	7.00 ± 0.14	18.45 ± 0.16	48.62 ± 0.28
Top-2	91.48% (2.00)	29.41% ± 0.60	71.57% ± 1.48	28.85 ± 0.17	7.59 ± 0.08	19.34 ± 0.14	52.39 ± 0.49
Top-3	86.93% (3.00)	34.80% ± 1.14	79.53% ± 0.35	30.69 ± 0.17	8.40 ± 0.11	20.64 ± 0.13	53.59 ± 0.35
Top-4	82.55% (4.00)	35.66% ± 0.30	81.13% ± 0.35	31.10 ± 0.10	8.53 ± 0.06	20.36 ± 0.11	54.96 ± 0.42
Top-5	78.13% (5.00)	39.09% ± 0.92	84.44% ± 0.46	31.16 ± 0.14	8.52 ± 0.08	20.69 ± 0.03	54.52 ± 0.13
Top-6	73.97% (6.00)	37.87% ± 0.90	83.70% ± 0.87	31.06 ± 0.04	8.38 ± 0.06	20.43 ± 0.08	56.18 ± 0.44
Neural Retrieval with Gemini API							
Top-1	95.99% (1.00)	34.80% ± 1.39	68.87% ± 0.62	27.86 ± 0.12	7.12 ± 0.04	18.76 ± 0.09	49.46 ± 0.23
Top-2	92.02% (2.00)	40.32% ± 0.92	81.50% ± 0.46	30.17 ± 0.08	8.03 ± 0.03	19.80 ± 0.08	55.48 ± 0.27
Top-3	87.93% (3.00)	40.93% ± 1.35	85.17% ± 1.25	31.36 ± 0.12	8.67 ± 0.10	20.68 ± 0.10	56.71 ± 0.27
Top-4	83.71% (4.00)	40.56% ± 0.62	84.31% ± 0.87	31.52 ± 0.11	8.59 ± 0.10	20.40 ± 0.10	56.47 ± 0.71
Top-5	79.47% (5.00)	40.20% ± 0.76	86.76% ± 0.60	31.32 ± 0.11	8.49 ± 0.11	20.49 ± 0.07	56.73 ± 0.91
Top-6	75.44% (6.00)	40.81% ± 0.52	87.01% ± 0.35	31.92 ± 0.02	8.73 ± 0.09	20.82 ± 0.05	58.39 ± 0.31
Truncated Raw Content							
First 6k words	32.59% (0.00)	14.71% ± 0.79	52.45% ± 0.69	25.42 ± 0.05	4.98 ± 0.09	16.58 ± 0.10	58.42 ± 0.11
Last 6k words	32.38% (0.00)	10.42% ± 0.62	35.66% ± 2.46	20.69 ± 0.19	3.44 ± 0.10	14.13 ± 0.08	44.23 ± 0.11
GistMem	83.13% (0.00)	40.20% ± 0.96	89.83% ± 0.76	31.00 ± 0.09	7.99 ± 0.04	20.15 ± 0.08	65.75 ± 0.20
ReadAgent-P							
Look up 1 pg	80.00% (0.98)	40.56% ± 0.46	89.46% ± 1.48	31.26 ± 0.09	8.22 ± 0.15	20.29 ± 0.07	63.78 ± 1.13
Look up 1-2 pgs	77.38% (1.71)	39.71% ± 1.87	89.71% ± 0.60	31.11 ± 0.04	8.01 ± 0.15	20.21 ± 0.04	64.73 ± 1.02
Look up 1-3 pgs	75.07% (2.53)	38.36% ± 1.21	89.71% ± 0.60	31.50 ± 0.29	8.15 ± 0.15	20.45 ± 0.24	63.91 ± 1.58
Look up 1-4 pgs	73.48% (3.08)	39.95% ± 1.51	90.56% ± 0.35	31.34 ± 0.05	8.08 ± 0.18	20.26 ± 0.07	63.40 ± 0.79
Look up 1-5 pgs	72.29% (3.50)	37.99% ± 0.96	87.75% ± 0.46	31.16 ± 0.10	8.06 ± 0.05	20.35 ± 0.12	65.22 ± 1.40
Look up 1-6 pgs	70.90% (3.97)	39.09% ± 2.04	88.24% ± 0.60	31.50 ± 0.30	8.05 ± 0.13	20.26 ± 0.13	66.70 ± 0.62
ReadAgent-S 1-6 pgs	70.34% (3.55)	46.57% ± 0.87	91.54% ± 0.30	32.90 ± 0.17	8.87 ± 0.23	21.15 ± 0.14	68.87 ± 0.60

Table 3: **QMSum validation** results (PaLM 2-L) means and standard deviations across 3 runs. 35 articles and 272 questions. **CR** is the compression rate. **# LU** is the number of lookups. **Resp. Length** is the length in words of the model’s final response.

values, which pushes down the F-Measures. Consequently, for the ROUGE scores to increase as text length increases, the improvement to recall must be more substantial than the reduction to precision. This happens to some extent, but the effect size is small. Furthermore, including gists in the text substantially increases the response length, as is the case for GistMem and all the ReadAgent approaches. This increase is in spite of the fact that all models use the same question-answering prompt, so there is no prompt difference to cause the increased response lengths. This makes it much more challenging for GistMem and ReadAgent to outperform the retrieval methods in ROUGE score. Nevertheless, ReadAgent-S manages to have the highest ROUGE scores as well as the highest LLM ratings. Because of these issues with ROUGE, we consider the LLM ratings to be more informative for comparisons between these runs. However, the LLM ratings do not make it easy to compare with results using a different LLM to rate, such as GPT, and they also do not allow for easy comparisons with other works. The same observation applies to the NarrativeQA results above.

4.6 ABLATION STUDY AND ANALYSIS

Retrieval Quality In Table 4, we compare using Gist-Mem with neural retrieval to look up one page with using ReadAgent to look up one page. This is equivalent to replacing ReadAgent’s prompt-based retrieval with neural retrieval. ReadAgent’s retrieval performs better here.

We provide additional ablation studies in Appendix A.

5 CONCLUSION

We have presented ReadAgent, a simple interactive prompting system to mitigate the context length and context use limitations of current LLMs. ReadAgent outperforms other strong zero-shot (i.e., not trained or finetuned on the training set) baselines across standard performance metrics. These results demonstrate that LLMs are capable of generating compressed textual representations of long contexts that are useful for tasks that humans think are important, even without knowing those tasks ahead of time. They also demonstrate that LLMs are capable of reasoning interactively over such compressed representations, using them to decide what information needs to be retrieved to effectively perform a known task. ReadAgent increases the effective context length by up to 20× while outperforming conventional retrieval techniques. However, it does not give infinite context lengths, nor does it guarantee good performance when the gist memory itself is extremely long. Future work will need to address these fundamental limitations in LLMs.

方法	压缩率 (# LU)	LLM评分-1	LLM评分-2	ROUGE-1	ROUGE-2	ROUGE-L	响应长度
BM25 检索							
Top-1	95.69% (1.00)	32.48% ± 1.65	63.85% ± 1.51	27.53 ± 0.23	7.00 ± 0.14	18.45 ± 0.16	48.62 ± 0.28
前两名	91.48% (2.00)	29.41% ± 0.60	71.57% ± 1.48	28.85 ± 0.17	7.59 ± 0.08	19.34 ± 0.14	52.39 ± 0.49
Top-3	86.93% (3.00)	34.80% ± 1.14	79.53% ± 0.35	30.69 ± 0.17	8.40 ± 0.11	20.64 ± 0.13	53.59 ± 0.35
Top-4	82.55% (4.00)	35.66% ± 0.30	81.13% ± 0.35	31.10 ± 0.10	8.53 ± 0.06	20.36 ± 0.11	54.96 ± 0.42
前五	78.13% (5.00)	39.09% ± 0.92	84.44% ± 0.46	31.16 ± 0.14	8.52 ± 0.08	20.69 ± 0.03	54.52 ± 0.13
Top-6	73.97% (6.00)	37.87% ± 0.90	83.70% ± 0.87	31.06 ± 0.04	8.38 ± 0.06	20.43 ± 0.08	56.18 ± 0.44
使用 Gemini API 进行神经检索							
Top-1	95.99% (1.00)	34.80% ± 1.39	68.87% ± 0.62	27.86 ± 0.12	7.12 ± 0.04	18.76 ± 0.09	49.46 ± 0.23
前两名	92.02% (2.00)	40.32% ± 0.92	81.50% ± 0.46	30.17 ± 0.08	8.03 ± 0.03	19.80 ± 0.08	55.48 ± 0.27
Top-3	87.93% (3.00)	40.93% ± 1.35	85.17% ± 1.25	31.36 ± 0.12	8.67 ± 0.10	20.68 ± 0.10	56.71 ± 0.27
Top-4	83.71% (4.00)	40.56% ± 0.62	84.31% ± 0.87	31.52 ± 0.11	8.59 ± 0.10	20.40 ± 0.10	56.47 ± 0.71
前五名	79.47% (5.00)	40.20% ± 0.76	86.76% ± 0.60	31.32 ± 0.11	8.49 ± 0.11	20.49 ± 0.07	56.73 ± 0.91
前6名	75.44% (6.00)	40.81% ± 0.52	87.01% ± 0.35	31.92 ± 0.02	8.73 ± 0.09	20.82 ± 0.05	58.39 ± 0.31
截断的原始内容							
前6千词	32.59% (0.00)	14.71% ± 0.79	52.45% ± 0.69	25.42 ± 0.05	4.98 ± 0.09	16.58 ± 0.10	58.42 ± 0.11
最后6千词	32.38% (0.00)	10.42% ± 0.62	35.66% ± 2.46	20.69 ± 0.19	3.44 ± 0.10	14.13 ± 0.08	44.23 ± 0.11
GistMem	83.13% (0.00)	40.20% ± 0.96	89.83% ± 0.76	31.00 ± 0.09	7.99 ± 0.04	20.15 ± 0.08	65.75 ± 0.20
ReadAgent-P							
查找1页	80.00% (0.98)	40.56% ± 0.46	89.46% ± 1.48	31.26 ± 0.09	8.22 ± 0.15	20.29 ± 0.07	63.78 ± 1.13
查找1-2页	77.38% (1.71)	39.71% ± 1.87	89.71% ± 0.60	31.11 ± 0.04	8.01 ± 0.15	20.21 ± 0.04	64.73 ± 1.02
查找1-3页	75.07% (2.53)	38.36% ± 1.21	89.71% ± 0.60	31.50 ± 0.29	8.15 ± 0.15	20.45 ± 0.24	63.91 ± 1.58
查找1-4页	73.48% (3.08)	39.95% ± 1.51	90.56% ± 0.35	31.34 ± 0.05	8.08 ± 0.18	20.26 ± 0.07	63.40 ± 0.79
查找1-5页	72.29% (3.50)	37.99% ± 0.96	87.75% ± 0.46	31.16 ± 0.10	8.06 ± 0.05	20.35 ± 0.12	65.22 ± 1.40
查找1-6页	70.90% (3.97)	39.09% ± 2.04	88.24% ± 0.60	31.50 ± 0.30	8.05 ± 0.13	20.26 ± 0.13	66.70 ± 0.62
ReadAgent-S 1-6 页	70.34% (3.55)	46.57% ± 0.87	91.54% ± 0.30	32.90 ± 0.17	8.87 ± 0.23	21.15 ± 0.14	68.87 ± 0.60

表3: **QMSum 验证集** 结果（PaLM 2-L）为3次运行的平均值和标准差。共35篇文章和272个问题。 **压缩率** 为压缩率。 **# 查找次数** 为查找次数。 **响应长度** 为模型最终响应的单词长度。

从而拉低了F值。因此，若要使ROUGE分数随文本长度增加而提升，召回率的改善必须比精确率的下降更为显著。这在某种程度上确实发生了，但效应量很小。此外，在文本中包含要点会显著增加响应长度，GistMem和所有ReadAgent方法都是如此。这种增加是在所有模型都使用相同问答提示的情况下发生的，因此不存在因提示差异导致响应长度增加的情况。这使得GistMem和ReadAgent在ROUGE分数上超越检索方法变得更具挑战性。尽管如此，ReadAgent-S仍设法获得了最高的ROUGE分数以及最高的LLM评分。由于ROUGE存在的这些问题，我们认为LLM评分对于这些实验之间的比较更具参考价值。然而，LLM评分不便于与使用不同LLM（例如GPT）进行评分的结果进行比较，也不便于与其他工作的结果进行直接比较。上述对NarrativeQA结果的观察同样适用。

4.6 消融研究与分析

检索质量在表4中，我们比较了使用Gist-Mem结合神经检索来查找一个页面与使用ReadAgent来查找一个页面的效果。这相当于将ReadAgent的基于提示的检索替换为神经检索。ReadAgent的检索在此表现更佳。

我们在附录A中提供了额外的消融研究。

5 结论

我们提出了ReadAgent，一个简单的交互式提示系统，旨在缓解当前大语言模型在上下文长度和上下文使用方面的限制。ReadAgent在标准性能指标上均优于其他强大的零样本（即未在训练集上进行训练或微调）基线。这些结果表明，大语言模型能够生成成长上下文的压缩文本表示，这些表示对于人类认为重要的任务是有用的，即使事先并不知道这些任务。它们还表明，大语言模型能够交互式地对这些压缩表示进行推理，利用它们来决定需要检索哪些信息以有效执行已知任务。ReadAgent将有效上下文长度提升了高达 20×，同时优于传统的检索技术。然而，它并不能提供无限的上下文长度，也无法保证在要点记忆本身极长时仍能保持良好的性能。未来的工作需要解决大语言模型中的这些根本性限制。

ACKNOWLEDGEMENTS

The authors thank Sergey Ioffe, Rif A. Saurous, Yujin Tang, Sergio Guadarrama, Daliang Li, Felix Yu, and Rob Fergus for valuable feedback and discussion.

REFERENCES

Ainslie, J., Lei, T., de Jong, M., Ontañón, S., Brahma, S., Zemlyanskiy, Y., Uthus, D., Guo, M., Lee-Thorp, J., Tay, Y., Sung, Y.-H., and Sanghai, S. Colt5: Faster long-range transformers with conditional computation, 2023.

Anil, R., Dai, A. M., Firat, O., Johnson, M., Lepikhin, D., Passos, A., Shakeri, S., Taropa, E., Bailey, P., Chen, Z., et al. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*, 2023.

Beltagy, I., Peters, M. E., and Cohan, A. Longformer: The long-document transformer, 2020.

Chen, D., Fisch, A., Weston, J., and Bordes, A. Reading wikipedia to answer open-domain questions, 2017.

Chen, H., Pasunuru, R., Weston, J., and Celikyilmaz, A. Walking down the memory maze: Beyond context limit through interactive reading. *arXiv preprint arXiv:2310.05029*, 2023a.

Chen, S., Wong, S., Chen, L., and Tian, Y. Extending context window of large language models via positional interpolation, 2023b.

Chen, Y., Qian, S., Tang, H., Lai, X., Liu, Z., Han, S., and Jia, J. Longlora: Efficient fine-tuning of long-context large language models. *arXiv preprint arXiv:2309.12307*, 2023c.

Chiang, C.-H. and Lee, H.-y. Can large language models be an alternative to human evaluations? In Rogers, A., Boyd-Graber, J., and Okazaki, N. (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15607–15631, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.870.

Chiang, W.-L., Li, Z., Lin, Z., Sheng, Y., Wu, Z., Zhang, H., Zheng, L., Zhuang, S., Zhuang, Y., Gonzalez, J. E., Stoica, I., and Xing, E. P. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023.

Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., and Su, Y. Mind2web: Towards a generalist agent for the web. *arXiv preprint arXiv:2306.06070*, 2023.

Dinan, E., Roller, S., Shuster, K., Fan, A., Auli, M., and Weston, J. Wizard of wikipedia: Knowledge-powered conversational agents, 2019.

Furuta, H., Matsuo, Y., Faust, A., and Gur, I. Language model agents suffer from compositional generalization in web automation. *arXiv preprint arXiv:2311.18751*, 2023.

Furuta, H., Lee, K.-H., Nachum, O., Matsuo, Y., Faust, A., Gu, S. S., and Gur, I. Multimodal web navigation with instruction-finetuned foundation models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=efFmBWioSc>.

Guo, M., Ainslie, J., Uthus, D., Ontanon, S., Ni, J., Sung, Y.-H., and Yang, Y. LongT5: Efficient text-to-text transformer for long sequences. In Carpuat, M., de Marneffe, M.-C., and Meza Ruiz, I. V. (eds.), *Findings of the Association for Computational Linguistics: NAACL 2022*, pp. 724–736, Seattle, United States, July 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.findings-naacl.55.

Gur, I., Furuta, H., Huang, A., Safdari, M., Matsuo, Y., Eck, D., and Faust, A. A real-world webagent with planning, long context understanding, and program synthesis. *arXiv preprint arxiv:2307.12856*, 2023.

致谢

作者感谢Sergey Ioffe、Rif A. Saurous、Yujin Tang、Sergio Guadarrama、Daliang Li、Felix Yu和Rob Fergus提供的宝贵反馈与讨论。

参考文献

Ainslie, J., Lei, T., de Jong, M., Ontañón, S., Brahma, S., Zemlyanskiy, Y., Uthus, D., Guo, M., Lee-Thorp, J., Tay, Y., Sung, Y.-H., and Sanghai, S. Colt5: 具有条件计算的更快长距离Transformer, 2023。Anil, R., Dai, A. M., Firat, O., Johnson, M., Lepikhin, D., Passos, A., Shakeri, S., Taropa, E., Bailey, P., Chen, Z., et al. Palm 2 技术报告。 *arXivpreprint arXiv:2305.10403*, 2023。Beltagy, I., Peters, M. E., and Cohan, A. Longformer: 长文档Transformer, 2020。Chen, D., Fisch, A., Weston, J., and Bordes, A. 阅读维基百科以回答开放域问题, 2017。Chen, H., Pasunuru, R., Weston, J., and Celikyilmaz, A. 漫步记忆迷宫: 通过交互式阅读超越上下文限制。 *arXivpreprint arXiv:2310.05029*, 2023a。Chen, S., Wong, S., Chen, L., and Tian, Y. 通过位置插值扩展大语言模型的上下文窗口, 2023b。Chen, Y., Qian, S., Tang, H., Lai, X., Liu, Z., Han, S., and Jia, J. Longlora: 长上下文大语言模型的高效微调。 *arXivpreprint arXiv:2309.12307*, 2023c。Chiang, C.-H. and Lee, H.-y. 大语言模型可以替代人类评估吗? In Rogers, A., Boyd-Graber, J., and Okazaki, N. (eds.), 第61届计算语言学协会年会论文集 (第一卷: 长论文), pp. 15607–15631, 加拿大多伦多, 2023年7月。计算语言学协会。doi: 10.18653/v1/2023.acl-long.870。Chiang, W.-L., Li, Z., Lin, Z., Sheng, Y., Wu, Z., Zhang, H., Zheng, L., Zhuang, S., Zhuang, Y., Gonzalez, J. E., Stoica, I., and Xing, E. P. Vicuna: 一款以90%* ChatGPT质量给GPT-4留下深刻印象的开源聊天机器人, 2023年3月。Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., and Su, Y. Mind2Web: 迈向通用网络智能体。 *arXivpreprint arXiv:2306.06070*, 2023。Dinan, E., Roller, S., Shuster, K., Fan, A., Auli, M., and Weston, J. 维基百科向导: 知识驱动的对话智能体, 2019。Furuta, H., Matsuo, Y., Faust, A., and Gur, I. 语言模型智能体在网络自动化中受困于组合泛化。 *arXivpreprint arXiv:2311.18751*, 2023。Furuta, H., Lee, K.-H., Nachum, O., Matsuo, Y., Faust, A., Gu, S. S., and Gur, I. 使用指令微调基础模型进行多模态网络导航。In 第十二届国际学习表征大会, 2024。URL <https://openreview.net/forum?id=efFmBWioSc>。Guo, M., Ainslie, J., Uthus, D., Ontanon, S., Ni, J., Sung, Y.-H., and Yang, Y. LongT5: 用于长序列的高效文本到文本Transformer。In Carpuat, M., de Marneffe, M.-C., and Meza Ruiz, I. V. (eds.), 计算语言学协会北美分会2022年会议发现集, pp. 724–736, 美国西雅图, 2022年7月。计算语言学协会。doi: 10.18653/v1/2022.findings-naacl.55。Gur, I., Furuta, H., Huang, A., Safdari, M., Matsuo, Y., Eck, D., and Faust, A. 一个具备规划、长上下文理解和程序合成能力的真实世界网络智能体。 *arXiv preprintarxiv:2307.12856*, 2023。

Han, C., Wang, Q., Xiong, W., Chen, Y., Ji, H., and Wang, S. Lm-infinite: Simple on-the-fly length generalization for large language models. *arXiv preprint arXiv:2308.16137*, 2023.

He, P., Liu, X., Gao, J., and Chen, W. DeBERTa: Decoding-enhanced bert with disentangled attention. *arXiv preprint arXiv:2006.03654*, 2020.

Izacard, G. and Grave, E. Leveraging passage retrieval with generative models for open domain question answering, 2021.

Jin, H., Han, X., Yang, J., Jiang, Z., Liu, Z., Chang, C.-Y., Chen, H., and Hu, X. Llm maybe longlm: Self-extend llm context window without tuning. *arXiv preprint arXiv:2401.01325*, 2024.

Kim, G., Baldi, P., and McAleer, S. Language models can solve computer tasks. *arXiv preprint arxiv:2303.17491*, 2023.

Kočiský, T., Schwarz, J., Blunsom, P., Dyer, C., Hermann, K. M., Melis, G., and Grefenstette, E. The narrativeqa reading comprehension challenge. *Transactions of the Association for Computational Linguistics*, 6:317–328, 2018.

Lanchantin, J., Toshniwal, S., Weston, J., Szlam, A., and Sukhbaatar, S. Learning to reason and memorize with self-notes. *arXiv preprint arXiv:2305.00833*, 2023.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

Lin, C.-Y. ROUGE: A package for automatic evaluation of summaries. In *Proceedings of the ACL Workshop: Text Summarization Branches Out 2004*, pp. 10, 01 2004.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023.

Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., et al. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.

Pang, R. Y., Parrish, A., Joshi, N., Nangia, N., Phang, J., Chen, A., Padmakumar, V., Ma, J., Thompson, J., He, H., et al. Quality: Question answering with long input texts, yes! In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 5336–5358, 2022.

Park, J. S., O’Brien, J., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–22, 2023.

Peng, B., Li, C., He, P., Galley, M., and Gao, J. Instruction tuning with gpt-4. *arXiv preprint arXiv:2304.03277*, 2023.

Press, O., Smith, N., and Lewis, M. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022.

Reyna, V. and Brainerd, C. Fuzzy-trace theory: Some foundational issues. *Learning and Individual differences*, 7(2):145–162, 1995a.

Reyna, V. F. A theory of medical decision making and health: fuzzy trace theory. *Medical decision making*, 28(6):850–865, 2008.

Reyna, V. F. A new intuitionism: Meaning, memory, and development in fuzzy-trace theory. *Judgment and Decision making*, 7(3):332–359, 2012.

Reyna, V. F. and Brainerd, C. J. Fuzzy-trace theory: An interim synthesis. *Learning and individual Differences*, 7(1):1–75, 1995b.

Robertson, S., Zaragoza, H., et al. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389, 2009.

Han, C., Wang, Q., Xiong, W., Chen, Y., Ji, H., and Wang, S. Lm-infinite: 大语言模型的简单即时长度泛化. *arXivpreprint arXiv:2308.16137*, 2023. He, P., Liu, X., Gao, J., and Chen, W. DeBERTa: 具有解耦注意力的解码增强型BERT. *arXiv preprintarXiv:2006.03654*, 2020. Izacard, G. and Grave, E. 利用生成模型进行段落检索以实现开放域问答, 2021. Jin, H., Han, X., Yang, J., Jiang, Z., Liu, Z., Chang, C.-Y., Chen, H., and Hu, X. LLM 也许是 LongLM: 无需调优即可自扩展LLM上下文窗口. *arXivpreprint arXiv:2401.01325*, 2024. Kim, G., Baldi, P., and McAleer, S. 语言模型可以解决计算机任务. *arXiv preprintarxiv:2303.17491*, 2023. Kořcisk`y, T., Schwarz, J., Blunsom, P., Dyer, C., Hermann, K. M., Melis, G., and Grefenstette, E. NarrativeQA阅读理解挑战赛. 计算语言学协会汇刊, 6:317–328, 2018. Lanchantin, J., Toshniwal, S., Weston, J., Szlam, A., and Sukhbaatar, S. 通过自注释学习推理和记忆. *arXiv preprint arXiv:2305.00833*, 2023. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. 面向知识密集型自然语言处理任务的检索增强生成. 神经信息处理系统进展, 33:9459–9474, 2020. Lin, C.-Y. ROUGE: 一个用于摘要自动评估的软件包. In *ACL 2004年文本摘要分支研讨会论文集*, pp. 10, 2004年1月. Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. 迷失在中间: 语言模型如何使用长上下文. *arXivpreprint arXiv:2307.03172*, 2023. Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., et al. Webgpt: 基于浏览器辅助问答与人类反馈. *arXivpreprintarXiv:2112.09332*, 2021. Pang, R. Y., Parrish, A., Joshi, N., Nangia, N., Phang, J., Chen, A., Padmakumar, V., Ma, J., Thompson, J., He, H., et al. QuALITY: 长输入文本问答, 是的! In *2022年计算语言学协会北美分会会议论文集: 人类语言技术*, pp. 5336–5358, 2022. Park, J. S., O’Brien, J., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. 生成式智能体: 人类行为的交互式模拟. In *第36届ACM用户界面软件与技术研讨会论文集*, pp. 1–22, 2023. Peng, B., Li, C., He, P., Galley, M., and Gao, J. 使用GPT-4进行指令微调. *arXiv preprintarXiv:2304.03277*, 2023. Press, O., Smith, N., and Lewis, M. 短训练, 长测试: 线性偏置注意力实现输入长度外推. In *国际学习表征大会*, 2022. Reyna, V. and Brainerd, C. 模糊痕迹理论: 一些基础性问题. 学习与个体差异, 7(2):145–162, 1995a. Reyna, V. F. 医疗决策与健康理论: 模糊痕迹理论. 医疗决策, 28(6):850–865, 2008. Reyna, V. F. 一种新的直觉主义: 模糊痕迹理论中的意义、记忆与发展. 判断与决策, 7(3):332–359, 2012. Reyna, V. F. and Brainerd, C. J. 模糊痕迹理论: 中期综述. 学习与个体差异, 7(1):1–75, 1995b. Robertson, S., Zaragoza, H., et al. 概率相关性框架: BM25及超越. 基础与趋势_{IR} 信息检索, 3(4):333–389, 2009.

Shaham, U., Segal, E., Ivgi, M., Efrat, A., Yoran, O., Haviv, A., Gupta, A., Xiong, W., Geva, M., Berant, J., et al. Scrolls: Standardized comparison over long language sequences. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 12007–12021, 2022.

Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E. H., Schärli, N., and Zhou, D. Large language models can be easily distracted by irrelevant context. In *International Conference on Machine Learning*, pp. 31210–31227. PMLR, 2023.

Shi, T., Karpathy, A., Fan, L., Hernandez, J., and Liang, P. World of bits: An open-domain platform for web-based agents. In *International Conference on Machine Learning*, 2017.

Sun, S., Liu, Y., Wang, S., Zhu, C., and Iyyer, M. Pearl: Prompting large language models to plan and execute actions over long documents. *arXiv preprint arXiv:2305.14564*, 2023.

Tay, Y., Dehghani, M., Bahri, D., and Metzler, D. Efficient transformers: A survey. *ACM Comput. Surv.*, 55(6), dec 2022. ISSN 0360-0300. doi: 10.1145/3530811. URL <https://doi.org/10.1145/3530811>.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. u., and Polosukhin, I. Attention is all you need. In Guyon, I., Luxburg, U. V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

Weston, J. and Sukhbaatar, S. System 2 attention (is something you might need too). *arXiv preprint arXiv:2311.11829*, 2023.

Wu, J., Ouyang, L., Ziegler, D. M., Stiennon, N., Lowe, R., Leike, J., and Christiano, P. Recursively summarizing books with human feedback. *arXiv preprint arXiv:2109.10862*, 2021.

Wu, Y., Rabe, M. N., Hutchins, D., and Szegedy, C. Memorizing transformers. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=TrjbxzRcnf->.

Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q. V., Zhou, D., and Chen, X. Large language models as optimizers. *arXiv preprint arXiv:2309.03409*, 2023.

Yang, K., Tian, Y., Peng, N., and Klein, D. Re3: Generating longer stories with recursive reprompting and revision. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 4393–4479, 2022.

Yao, S., Chen, H., Yang, J., and Narasimhan, K. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022.

Yuan, X., Fu, J., Côté, M.-A., Tay, Y., Pal, C., and Trischler, A. Interactive machine comprehension with information seeking agents. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 2325–2338, 2020.

Zaheer, M., Guruganesh, G., Dubey, K. A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., and Ahmed, A. Big bird: Transformers for longer sequences. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 17283–17297. Curran Associates, Inc., 2020.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023.

Shaham, U., Segal, E., Ivgi, M., Efrat, A., Yoran, O., Haviv, A., Gupta, A., Xiong, W., Geva, M., Berant, J., 等人。SCROLLS: 长语言序列标准化比较。载于2022年自然语言处理实证方法会议论文集，第12007–12021页，2022。Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E. H., Schärli, N., 和 Zhou, D. 大语言模型容易被无关上下文干扰。载于国际机器学习大会，第31210–31227页。PMLR, 2023。Shi, T., Karpathy, A., Fan, L., Hernandez, J., 和 Liang, P. 比特世界：一个基于网络的开放领域代理平台。载于国际机器学习大会，2017。Sun, S., Liu, Y., Wang, S., Zhu, C., 和 Iyyer, M. PEARL: 提示大语言模型对长文档进行规划与执行。*arXiv*预印本 *arXiv:2305.14564*, 2023。Tay, Y., Dehghani, M., Bahri, D., 和 Metzler, D. 高效Transformer: 综述。ACM计算概览, 55(6), 2022年12月。ISSN 0360-0300. doi: 10.1145/3530811. URL <https://doi.org/10.1145/3530811>. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. u., 和 Polosukhin, I. 注意力机制就是一切。载于 Guyon, I., Luxburg, U. V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., 和 Garnett, R. (编), 神经信息处理系统进展, 第30卷。Curran Associates, Inc., 2017。Weston, J. 和 Sukhbaatar, S. 系统2注意力（或许你也需要）。*arXiv*预印本 *arXiv:2311.11829*, 2023。Wu, J., Ouyang, L., Ziegler, D. M., Stiennon, N., Lowe, R., Leike, J., 和 Christiano, P. 基于人类反馈的递归书籍摘要。*arXiv*预印本 *arXiv:2109.10862*, 2021。Wu, Y., Rabe, M. N., Hutchins, D., 和 Szegedy, C. 记忆Transformer。载于国际学习表征大会，2022。URL <https://openreview.net/forum?id=TrjbxzRcnf->. Xiao, G., Tian, Y., Chen, B., Han, S., 和 Lewis, M. 具有注意力汇聚机制的高效流式语言模型。*arXiv*预印本 *arXiv:2309.17453*, 2023。Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q. V., Zhou, D., 和 Chen, X. 作为优化器的大语言模型。*arXiv*预印本 *arXiv:2309.03409*, 2023。Yang, K., Tian, Y., Peng, N., 和 Klein, D. Re3: 通过递归重新提示与修订生长故事。载于2022年自然语言处理实证方法会议论文集，第4393–4479页，2022。Yao, S., Chen, H., Yang, J., 和 Narasimhan, K. WebShop: 基于具身语言代理的可扩展现实世界网络交互。神经信息处理系统进展, 35:20744–20757, 2022。Yuan, X., Fu, J., Côté, M.-A., Tay, Y., Pal, C., 和 Trischler, A. 基于信息寻求代理的交互式机器理解。载于第58届计算语言学协会年会论文集，第2325–2338页，2020。Zaheer, M., Guruganesh, G., Dubey, K. A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., 和 Ahmed, A. Big Bird: 用于长序列的Transformer。载于 Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., 和 Lin, H. (编), 神经信息处理系统进展, 第33卷, 第17283–17297页。Curran Associates, Inc., 2020。Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. E., 和 Stoica, I. 使用MT-bench和聊天机器人竞技场评判LLM作为裁判。载于第三十七届神经信息处理系统会议数据集与基准测试赛道, 2023。

Zhong, M., Yin, D., Yu, T., Zaidi, A., Mutuma, M., Jha, R., Hassan, A., Celikyilmaz, A., Liu, Y., Qiu, X., et al. Qmsum: A new benchmark for query-based multi-domain meeting summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 5905–5921, 2021.

Zhong, W., Guo, L., Gao, Q., and Wang, Y. Memorybank: Enhancing large language models with long-term memory. *arXiv preprint arXiv:2305.10250*, 2023.

Zhong, M., Yin, D., Yu, T., Zaidi, A., Mutuma, M., Jha, R., Hassan, A., Celikyilmaz, A., Liu, Y., Qiu, X., 等人。QMSum：基于查询的多领域会议摘要新基准。载于2021年北美计算语言学协会年会：人类语言技术会议论文集，第5905–5921页，2021。

Zhong, W., Guo, L., Gao, Q., and Wang, Y. Memorybank：为大型语言模型增强长期记忆。*arXiv*预印本 *arXiv:2305.10250*，2023。

A ABLATION STUDY AND ANALYSIS (CONTINUED)

Episode pagination In this work we ask ReadAgent to decide where to pause reading and what information to store together in memory (Section 3.1), whereas in prior art, rule-based segmentation of text is typically used Chen et al. (2023a); Wu et al. (2021). We compare the two approaches with similar page length on average in Table 5 to demonstrate that it is indeed beneficial to break at pause points that LLMs consider natural (e.g. scene transitions, ends of dialogue, narrative transitions, etc).

	LLM	Uniform Length
ReadAgent-P (1-5 pgs) Acc.	86.83%	85.71%

Table 5: ReadAgent accuracy on QuALITY with episode pagination based on LLM (PaLM 2-L) vs. uniform length pagination.

The compression trade-off Table 6 presents the empirical results of compression rate increasing as page size increases. As the compression rate decreases, the gists are more useful for answering questions directly. However, for ReadAgent with look-ups, when the initial gist compression rate gets too high, accuracy suffers.

max_words	GistMem		ReadAgent-P (1-5 pgs)	
	CR	Acc	CR	Acc
400	81.81%	78.91%	66.71%	86.82%
600	85.53%	77.52%	66.45%	86.83%
800	88.12%	76.22%	65.06%	86.34%
1200	91.38%	73.97%	61.77%	85.67%

Table 6: Compression rate increases as the maximum number of words allowed per page increases on QuALITY. Our default setting of min/max words is 280/600. In the other three experiments, we scale min words proportionally with max words.

B EVALUATION WITH GPT-3.5

Table 7 shows the results of running experiments using exactly the same setup as described in Section 4.3, but using GPT 3.5 Turbo rather than PaLM 2-L. GPT 3.5 Turbo has a context length of over 16,000 tokens, so the QuALITY dataset easily fits into context. We don’t specifically tune prompts for GPT 3.5 Turbo, but instead use the same prompts that we use for PaLM 2-L. GPT 3.5 Turbo has a much harder time with this task than PaLM 2-L, but the same general trends hold. Neural Retrieval is weaker than ReadAgent. ReadAgent-S achieves comparable performance to using the full article content. The gap between ReadAgent-P and ReadAgent-S appears to be larger using this model, but we found that ReadAgent-P is very restrictive of how many pages to look up (1.0 in average) even though we allow up to 5. We think that this can likely be remedied if we engineer the prompt for GPT 3.5 Turbo. Nonetheless, comparing to using top 3 from neural retrieval, ReadAgent-P still yields better accuracy and compression rate.

Method	CR (# LU)	Accuracy
Neural Retrieval with Gemini API Top-3	73.13% (3)	69.22%
Full Raw Content	0%	73.30%
GistMem	84.24%	66.06%
ReadAgent-P Look up 1-5 pgs	76.60% (1.0)	69.65%
ReadAgent-S Look up 1-6 pgs	60.43% (3.4)	72.10%

Table 7: QuALITY results on the dev set of 230 docs and 2086 questions using GPT-3.5-turbo. **CR** is the compression rate. **# LU** is the number of lookups. We report 1 run for each experiment for cost considerations.

A 消融研究与分析（续）

片段分页在本工作中，我们要求ReadAgent决定在何处暂停阅读以及将哪些信息共同存储在记忆中（第3.1节），而在现有技术中，通常使用基于规则的分割文本方法 Chen等人 (2023a); Wu et al. (2021)。我们在表5中比较了这两种平均页面长度相近的方法，以证明在大语言模型认为自然的暂停点（例如场景转换、对话结束、叙事转换等）进行中断确实是有益的。

	LLM	均匀长度
ReadAgent-P (1-5页) 准确率	86.83%	85.71%

表5：基于大语言模型（PaLM 2-L）的片段分页与均匀长度分页下，ReadAgent在QuALITY数据集上的准确率。

压缩权衡表6展示了压缩率随页面大小增加而提高的经验性结果。随着压缩率降低，要点对于直接回答问题更为有用。然而，对于具备查找功能的ReadAgent，当初始要点压缩率过高时，准确率会受到影响。

最大词数	GistMem		ReadAgent-P (1-5 页)	
	CR	Acc	CR	Acc
400	81.81%	78.91%	66.71%	86.82%
600	85.53%	77.52%	66.45%	86.83%
800	88.12%	76.22%	65.06%	86.34%
1200	91.38%	73.97%	61.77%	85.67%

表6：在QuALITY数据集上，随着每页面允许的最大词数增加，压缩率也随之提升。我们的默认最小词数/最大词数设置为280/600。在另外三个实验中，我们按比例调整最小词数以匹配最大词数。

B 使用GPT-3.5进行评估

表7展示了使用与第4节第3小节所述完全相同的实验设置，但使用GPT 3.5 Turbo而非PaLM 2-L运行实验的结果。GPT 3.5 Turbo的上下文长度超过16,000个词元，因此QuALITY数据集可以轻松放入上下文。我们没有专门为GPT 3.5 Turbo调整提示词，而是使用了与PaLM 2-L相同的提示词。GPT 3.5 Turbo在此任务上的表现比PaLM 2-L困难得多，但总体趋势保持一致。神经检索弱于ReadAgent。ReadAgent-S实现了与使用完整文章内容相当的性能。使用此模型时，ReadAgent-P和ReadAgent-S之间的差距似乎更大，但我们发现ReadAgent-P对查找多少页面限制非常严格（平均1.0），尽管我们允许最多5个。我们认为，如果为GPT 3.5 Turbo设计提示词，这个问题很可能得到解决。尽管如此，与使用神经检索的前3个结果相比，ReadAgent-P仍然能产生更好的准确率和压缩率。

方法	压缩率 (# LU)	准确率
使用 Gemini API Top-3 的神经检索	73.13% (3)	69.22%
完整原始内容	0%	73.30%
GistMem	84.24%	66.06%
ReadAgent-P 查阅 1-5 页	76.60% (1.0)	69.65%
ReadAgent-S 查找 1-6 页	60.43% (3.4)	72.10%

表 7：使用 GPT-3.5-turbo 在包含 230 篇文档和 2086 个问题的开发集上的 QuALITY 结果。**压缩率**是压缩率。**# 查找次数** 是查找次数。出于成本考虑，我们为每个实验报告了 1 次运行结果。

C CASE STUDY

In this section, we analyze reading comprehension examples to demonstrate where the ability to simultaneously think over long-range global context and focus on local information is important. We selected the short story “off course” by Mack Reynolds⁴ because it is extremely short (2,712 words) and it is only broken into 8 pages, yet even so, neural retrieval using 4 pages gets three questions wrong that ReadAgent correctly answers. For this story, ReadAgent answers 6 of 8 questions correctly. Neural retrieval answers 3 of 8 correctly, and doesn’t get either question correct that ReadAgent misses. Note that in all three examples, ReadAgent only chooses to select two pages, even though it is also permitted to select up to 4. This flexibility is another advantage that ReadAgent has over standard retrieval systems.

“off course” Gist Memory

⟨P0⟩ Patrolmen Dermott and Casey encounter Dameri Tass, an alien who has landed on Earth. Dameri attempts to communicate with them using a device that translates his thoughts into English.

⟨P1⟩ The alien Dameri Tass used a helmet to learn English from Tim Casey, an Irish patrolman. He then became fascinated by a horse and wanted to use the helmet on the animal. Patrolman Dermott felt like he was in a shaggy dog story.

⟨P2⟩ A helicopter arrived, interrupting the horse’s inspection. Two Army officers exited and ordered a police cordon around the spacecraft. The alien spoke, surprising the general. More police and military personnel arrived.

⟨P3⟩ Dameri Tass, an alien visitor, was whisked away to Washington and held incommunicado for several days. His arrival caused a global furor. Officials worried about the potential impact of his message on society. Eventually, the UN demanded that he be allowed to speak before the Assembly. The White House agreed and a date was set.

⟨P4⟩ The world eagerly awaited a message from space. Dameri Tass, an envoy from a super-civilization, was expected to guide the world. Most people were ready to be guided, but some were not. The U.N. Secretary-General was nervous about introducing the envoy, as they knew very little about him. He had been asleep for most of his time on Earth and had only recently woken up. He spent his time playing with a dog, cat, and mouse. The Secretary-General was worried about what the envoy would say.

⟨P5⟩ Dameri Tass, an alien, is brought to Earth and mistaken for an envoy from another planet. He reveals he is just a collector for a zoo.

⟨P6⟩ Dameri Tass, an alien, mistakenly landed on Earth. He addressed a large crowd, criticizing their weapons, wars, and lack of a planet-wide government. He then left, refusing to take any Earth creatures with him, but expressing interest in horses.

⟨P7⟩ The others watched as the first visitor from space hurriedly left Earth.

Page #	Starting sentence in text
0	Shure and begorra...
1	The alien stooped down...
2	Interest in the horse was ended...
3	“Sure, and it’s quite a reception”...
4	Excitement, anticipation...
5	“Here he comes,”...
6	He straightened and started off...
7	The others drew back...

Table 8: Pagination for “off course”.

Distracting retrieval The first question gives an example of retrieval of distracting pages and the lack of global context provided by the gist memory causing the LLM to select the incorrect answer when using neural retrieval, even though it had also retrieved the pages that should have led to the correct answer. We provide the gist memory below and the story’s pagination in Table 8.

⁴Available at <http://aleph.gutenberg.org/3/0/0/3/30035//30035-h//30035-h.htm>.

C 案例研究

在本节中，我们分析阅读理解示例，以展示同时思考长距离全局上下文和聚焦局部信息的能力在何处至关重要。我们选择了Mack Reynolds的短篇小说《偏离航线》⁴，因为它极短（2,712字），且仅分为8个页面，但即便如此，使用4个页面的神经检索仍答错了三个ReadAgent正确回答的问题。对于这个故事，ReadAgent正确回答了8个问题中的6个。神经检索正确回答了8个问题中的3个，并且ReadAgent答错的两个问题，神经检索也未能答对。请注意，在所有三个示例中，ReadAgent仅选择两个页面，尽管它也被允许选择最多4个。这种灵活性是ReadAgent相对于标准检索系统的另一个优势。

“偏离航线” 要点记忆

⟨P0⟩ 巡警 Dermott 和 Casey 遇到了降落在地球的外星人达梅里·塔斯。达梅里试图用一个能将其思想翻译成英语的设备与他们交流。⟨P1⟩ 外星人达梅里·塔斯使用一个头盔向爱尔兰巡警 Tim Casey 学习英语。随后，他对一匹马产生了浓厚兴趣，并想对这只动物使用头盔。巡警 Dermott 感觉自己仿佛置身于一个冗长无聊的故事中。⟨P2⟩ 一架直升机抵达，打断了检查马匹的过程。两名陆军军官走下飞机，下令在航天器周围设立警戒线。外星人开口说话，令将军大吃一惊。更多警察和军事人员抵达现场。⟨P3⟩ 外星访客达梅里·塔斯被迅速带往华盛顿，并被隔离数日，无法与外界联系。他的到来引发了全球轰动。官员们担心他的信息可能对社会造成影响。最终，联合国要求允许他在联合国大会上发言。白宫表示同意并确定了日期。⟨P4⟩ 世界热切期待着来自太空的信息。来自超级文明的使者达梅里·塔斯，被期望能引导世界。大多数人已准备好接受引导，但有些人则不然。联合国秘书长对介绍这位使者感到紧张，因为人们对他的了解甚少。他在地球上的大部分时间都在沉睡，最近才醒来。他醒着的时候都在和一只狗、一只猫和一只老鼠玩耍。秘书长担心这位使者会说些什么。⟨P5⟩ 外星人达梅里·塔斯被带到地球，并被误认为是来自另一颗星球的使者。他透露自己只是一个动物园的收藏家。⟨P6⟩ 外星人达梅里·塔斯误降在地球上。他向一大群人发表讲话，批评了他们的武器、战争以及缺乏全球政府。随后他离开了，拒绝带走任何地球生物，但对马匹表示了兴趣。⟨P7⟩ 其他人目睹了第一位来自太空的访客匆匆离开地球。

页码	起始句
0	Shure and begorra...
1	外星人弯下腰...
2	对马的兴趣就此终结...
3	“当然，这真是盛大的接待” ...
4	兴奋，期待……
5	“他来了。”...
6	他挺直身子，开始动身……
7	其他人向后退去……

表8：《偏离航线》的分页信息。

干扰性检索 第一个问题给出了一个干扰性页面检索的例子，以及要点记忆缺乏全局上下文，导致大语言模型在使用神经检索时选择了错误答案，尽管它也检索到了本应指向正确答案的页面。我们在下方提供了要点记忆，并在表8中提供了故事的分页信息。

⁴可在 <http://aleph.gutenberg.org/3/0/0/3/30035//30035-h//30035-h.htm> 获取。

“off course” Question 1

What was Dameri’s purpose in landing on earth?
(A) He wanted to witness an uncivilized planet and share knowledge
(B) His spaceship needed to land for repairs
(C) He heard reports that Earth had interesting animal specimens for his collection
(D) He arrived on accident while exploring planets in the Galactic League
The correct answer is (D). ReadAgent chose (D). Neural retrieval chose (C).

For the question above, ReadAgent looked up pages 5 and 6. Neural retrieval looked up pages 3, 4, 5, and 6. Pages 4 and 5 both make prominent mention of animals, and Page 5 explicitly mentions that the alien is a collector for a zoo, so answer (C) seems reasonable based on the information on those pages. However, Pages 5 and 6, together with the global context from the gist memory, make it clear that (D) is the correct answer. Since neural retrieval provided both of those pages, the lack of the global context combined with the additional distractor pages led the LLM astray.

“off course” Question 2

What happened to Dameri while he was in custody of the government?
(A) He picked up an accent from the guards
(B) He slept almost the entire time
(C) He learned horses were creatures that could be ridden
(D) He was too shy to speak
The correct answer is (B). ReadAgent chose (B). Neural retrieval chose (A).

Incorrect retrieval The same story provides two examples of the consequences of incorrect retrieval, and the benefits of the gist memory. For the question above, ReadAgent looked up pages 3 and 4. Neural retrieval looked up pages 0, 1, 3, and 6. The correct answer is clearly stated on Page 4, and also clearly stated in the gist of Page 4. If the LLM had access to either of those, it should have been able to answer correctly. Instead, it was undoubtedly confused by Pages 0 and 1, where the alien learns an accent from one of the police officers in the initial encounter.

“off course” Question 3

How did Dameri Tass communicate in English?
(A) He could communicate telepathically
(B) He never was able to communicate in English
(C) He used a handheld translation device
(D) He acquired the knowledge from a human
The correct answer is (D). ReadAgent chose (D). Neural retrieval chose (C).

For the question above, ReadAgent looked up pages 0 and 1. Neural retrieval looked up pages 0, 3, 4, and 6. The critical information was in Page 1, although Page 0 was also relevant. The remaining pages were only relevant in that they demonstrated that (B) was incorrect. Again, the gist memory was sufficient to answer the question correctly, in addition to providing clear signal about what pages are relevant to the question. But neural retrieval’s selection of Page 0 without Page 1 made (C) seem plausible, as Page 0 discusses a device that the alien was clearly trying to use for communication.

D READAGENT FOR WEB NAVIGATION

We made an attempt to extend ReadAgent to decision making tasks. In particular, we apply ReadAgent for autonomous *web navigation* (Shi et al., 2017; Kim et al., 2023; Furuta et al., 2024), where the goal is to autonomously control browsers or computer interfaces to complete tasks with natural language instructions provided by users. Such instruction would be something like *Book an appointment for applying new passport for one adult, Ellen Walker, with phone number 123-456-7890 and email address EW@gmail.com on April 4, 2023 at 1 pm in the post office nearest to zip code 60505. Don’t send updates via text message*. Example web agent actions include *click*, *type*, and *select* (e.g. *click*,

“偏离航线” 问题 1

达梅里登陆地球的目的是什么？ (A) 他想亲眼目睹一个未开化的星球并分享知识 (B) 他的飞船需要降落进行维修 (C) 他听说地球上有趣味的动物标本可供他收藏 (D) 他在探索银河联盟的行星时意外抵达 正确答案是 (D)。ReadAgent 选择了 (D)。神经检索选择了 (C)。

对于上述问题，ReadAgent 查阅了第 5 页和第 6 页。神经检索查阅了第 3、4、5 和 6 页。第 4 页和第 5 页都突出提到了动物，并且第 5 页明确提到外星人是一个动物园的收藏家，因此根据这些页面上的信息，答案 (C) 似乎是合理的。然而，结合来自要点记忆的全局上下文，第 5 页和第 6 页清楚地表明 (D) 才是正确答案。由于神经检索提供了这两个页面，缺乏全局上下文加上额外的干扰页面导致大语言模型得出了错误的结论。

“偏离航线” 问题 2

达梅里在政府关押期间发生了什么？ (A) 他从守卫那里学来了口音 (B) 他几乎一直在睡觉 (C) 他了解到马是可以骑乘的生物 (D) 他太害羞而不敢说话 正确答案是 (B)。ReadAgent 选择了 (B)。神经检索选择了 (A)。

错误的检索 同一个故事提供了两个例子，说明错误检索的后果以及要点记忆的好处。对于上述问题，ReadAgent 查阅了第 3 页和第 4 页。神经检索查阅了第 0、1、3 和 6 页。正确答案在第 4 页有明确说明，在第 4 页的要点中也有明确说明。如果大语言模型能够访问其中任何一个，它都应该能够正确回答。然而，它无疑被第 0 页和第 1 页的内容所迷惑，那里描述了外星人在初次遭遇时从一名警察那里学会了口音。

“偏离航线” 问题 3

达梅里·塔斯是如何用英语交流的？ (A) 他可以通过心灵感应交流 (B) 他从未能够用英语交流 (C) 他使用手持翻译设备 (D) 他从人类那里获取了知识。 正确答案是 (D)。ReadAgent 选择了 (D)。神经检索选择了 (C)。

对于上述问题，ReadAgent 查找了页面 0 和 1。神经检索则查找了页面 0、3、4 和 6。关键信息位于页面 1 中，尽管页面 0 也相关。其余页面仅在于它们证明了选项 (B) 是错误的这一点上相关。同样，要点记忆足以正确回答问题，同时还清晰地指示了哪些页面与问题相关。但神经检索选择了页面 0 而未选择页面 1，这使得选项 (C) 看起来似乎合理，因为页面 0 讨论了一个外星人显然试图用于通信的设备。

D 用于网络导航的ReadAgent

我们尝试将ReadAgent扩展到决策任务中。具体而言，我们将ReadAgent应用于自主网络导航（Shi等人，2017；Kim等人，2023；Furuta等人，2024），其目标是自主控制浏览器或计算机界面，以完成用户提供的自然语言指令所指定的任务。此类指令可能类似于为一位成人Ellen Walker预约办理新护照，电话号码为123-456-7890，电子邮件地址为EW@gmail.com，时间为2023年4月4日下午1点，地点为最靠近邮编60505的邮局。不要通过短信发送更新通知）。网页智能体操作的示例包括点击、输入和选择（例如点击，

type nearest post office, select April 4, 2023). Because real-world websites can have very long HTML, LLM web agents often struggle with context length if it operates on raw content (Gur et al., 2023).

D.1 IMPLEMENTATION

Pagination For HTML, we leverage the explicit HTML DOM tree structure, decomposing the HTML into snippets with elements at a target depth and their descendants. We test the depth from 5 to 7 and choose the best. We use these snippets as the “pages” instead of asking the LLM to paginate.

Memory Gisting Similar to ReadAgent for reading comprehension, we prompt the LLM to summarize snippets into gists zero-shot, and subsequently concatenate the gists. We contextualize the gists with snippet index number in a python dictionary-format (e.g. {“index”: ..., “content”: ...}).

Interactive Look-up In the interactive look-up step, the LLM looks at a given task instruction, previous action history, and the gists to decide which original HTML snippets it wants to look up. We experimented with parallel look-up (ReadAgent-P) in the web navigation setting for faster experiments. Finally, to predict next-step actions, the LLM reads the retrieved snippets again and predicts the target element id to interact with, the type of action operation (click, type or select), and the input value (if any).

D.2 MIND2WEB

We evaluate ReadAgent for Web Navigation on the Mind2Web (Deng et al., 2023) dataset, a real-world planning and web action prediction benchmark, consisting of 2K instructions and episodes collected from 137 websites. The agent’s task is to predict the next-step action (click, type and select) given HTML, task instruction, and previous action history. Mind2Web has three test set splits: cross-task (252 tasks from 69 websites), cross-website (177 tasks from 10 websites), and cross-domain (912 tasks from 73 websites), which was originally designed for different testing different type of generalization. However, since our approach is zero-shot without training, these splits do not serve their original purposes.

Baselines MindAct from the Mind2Web paper (Deng et al., 2023) first uses a DeBERTa-base (He et al., 2020) model trained for task-relevant element retrieval to get the top 50 relevant elements. Instead of directly predicting target element id (part of an action), it formulates this task as iterative multi-choice question-answering with target element ids sampled from the top 50 and uses the LLM to solve it for performance purpose (see Deng et al. (2023) for details). The same LLM also predicts the type of action and an optional value. MindAct (GPT-4) results are the state-of-the-art. We additional generate MindAct results with PaLM 2-L as a reference.

Following the reading comprehension experiments (Section 4), we also compare with using full raw HTML, retrieval with BM25, neural retrieval with Gemini API embedding model (models/embedding-001), and using the gists without look-up, which, like ReadAgent, are not trained for web navigation tasks. We ask the LLM to directly predict that target element id as it is a simpler and more tractable implementation in our setting.

D.3 RESULTS

As shown in Table 9, ReadAgent achieves strong performance compared to the baselines. In particular, the results are even better than MindAct (PaLM 2-L), which uses the supervisedly learned Rank LM, despite ReadAgent not using models trained on the web navigation domain. Prior work shows that state-of-the-art LLMs alone are generally still weaker than the approaches using models specifically trained for the web navigation domain (Furuta et al., 2023).

Figure 9 shows that gisting effectively reduces the number input tokens. Most of the input gists require less than 8K tokens. For example, 97.4% of gisted inputs in cross-website split fits into the 8k context length, while only 51.5% of raw HTML can fit in the context window. The inputs are truncated for the parts that exceed the context length limit, which can significantly impact performance.

The results in Figure 9 and Table 9 indicate that even using the gist memory and ReadAgent retrieval causes truncation on many web pages. This is because the retrieved snippets are quite large, causing

输入最近邮局、选择2023年4月4日）。由于现实世界的网站可能包含非常长的HTML，如果大语言模型网页智能体直接处理原始内容，通常会受限于上下文长度（Gur等人，2023）。

D.1 实现

分页对于HTML，我们利用其显式的HTML DOM树结构，将HTML分解为包含目标深度元素及其后代的片段。我们测试了深度5到7，并选择最佳方案。我们将这些片段作为“页面”使用，而不是让大语言模型进行分页。

记忆要点提取与用于阅读理解的ReadAgent类似，我们提示大语言模型以零样本方式将片段总结为要点，随后将这些要点串联起来。我们使用Python字典格式（例如 { “index” : ..., “content” : ...}）将要点与片段索引号关联起来。

交互式查找在交互式查找步骤中，大语言模型会查看给定的任务指令、先前的操作历史以及要点，以决定它想要查找哪些原始HTML片段。我们在网络导航设置中尝试了并行查找（ReadAgent-P）以加速实验。最后，为了预测下一步操作，大语言模型会再次读取检索到的片段，并预测要交互的目标元素ID、操作类型（点击、输入或选择）以及输入值（如果有的话）。

D.2 MIND2WEB

我们在Mind2Web（Deng等人，2023）数据集上评估ReadAgent的网络导航能力。该数据集是一个真实世界的规划和网络动作预测基准，包含从137个网站收集的2K条指令和交互片段。智能体的任务是在给定HTML、任务指令和先前动作历史的情况下，预测下一步动作（点击、输入和选择）。Mind2Web包含三个测试集划分：跨任务（来自69个网站的252个任务）、跨网站（来自10个网站的177个任务）和跨领域（来自73个网站的912个任务），这些划分最初旨在测试不同类型的泛化能力。然而，由于我们的方法是零样本且无需训练，这些划分并未发挥其原始作用。

基线来自Mind2Web论文（Deng等人，2023）的MindAct首先使用一个为任务相关元素检索而训练的DeBERTa-base（He等人，2020）模型来获取前50个相关元素。它并非直接预测目标元素ID（动作的一部分），而是将此任务表述为迭代式多项选择问答，目标元素ID从前50个候选中采样，并利用大语言模型来解决它以提升性能（详见Deng等人（2023））。同一个大语言模型还预测动作类型和可选值。MindAct（GPT-4）的结果是当前最先进的。我们额外生成了使用PaLM 2-L的MindAct结果作为参考。

继阅读理解实验（第4节）之后，我们还与使用完整原始HTML、BM25检索、采用 Gemini API嵌入模型（models/embedding-001）的神经检索，以及使用无需查找的要点（与ReadAgent一样，未针对网页导航任务进行训练）进行了比较。我们要求大语言模型直接预测目标元素ID，因为这在我们的设置中是一种更简单且更易处理的实现方式。

D.3 结果

如表9所示，与基线相比，ReadAgent取得了强劲的性能。特别是，其结果甚至优于使用监督学习Rank LM的MindAct（PaLM 2-L），尽管ReadAgent并未使用在网页导航领域训练的模型。先前的研究表明，仅凭最先进的大语言模型通常仍弱于专门针对网页导航领域训练的模型所采用的方法（Furuta等人，2023）。

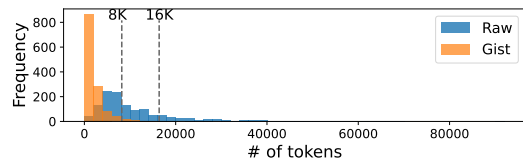
图9显示，要点提取有效减少了输入词元数量。大多数输入要点所需的词元数少于8K。例如，在跨网站分割中，97.4%的要点化输入能适配8k上下文长度，而仅有51.5%的原始HTML能适配上下文窗口。超出上下文长度限制的部分输入会被截断，这可能显著影响性能。

图9和表9中的结果表明，即使使用要点记忆和ReadAgent检索，也会导致许多网页被截断。这是因为检索到的片段相当大，导致

	Cross-Task					Cross-Website					Cross-Domain				
	CR	Ele. Acc	Op. F1	Step SR	SR	CR	Ele. Acc	Op. F1	Step SR	SR	CR	Ele. Acc	Op. F1	Step SR	SR
Using supervisedly trained RankLM															
MindAct (GPT-3.5 + Rank LM*)	–	20.3	56.6	17.4	0.8	–	19.3	48.8	16.2	0.6	–	21.6	52.8	18.6	1.0
MindAct (GPT-4 + Rank LM*)	–	41.6	60.6	36.2	2.0	–	35.8	51.1	30.1	2.0	–	37.1	46.5	26.4	2.0
MindAct (PaLM 2-L + Rank LM*)	–	29.8	61.9	24.4	1.2	–	28.8	59.6	21.6	0.6	–	29.9	60.4	24.5	1.3
No training															
(PaLM 2-L)															
+Raw HTML	0.0	22.1	<u>76.7</u>	19.2	1.2	0.0	22.2	72.3	18.2	1.7	0.0	23.6	75.6	20.9	1.0
+BM25 Retrieval (Top-1)	43.7	16.3	61.7	14.2	0.4	49.7	17.8	60.8	15.2	0.0	51.6	17.3	60.4	15.9	0.0
+BM25 Retrieval (Top-5)	19.5	25.9	70.4	22.4	2.0	17.6	29.5	71.8	23.1	1.7	19.2	27.6	71.1	24.4	1.0
+Neural Retrieval (Top-1)	74.4	14.6	55.5	11.7	0.4	87.9	18.0	55.8	14.0	0.0	82.8	16.4	60.3	14.2	0.1
+Neural Retrieval (Top-5)	32.4	26.4	71.9	22.6	0.8	37.2	26.7	69.1	22.3	2.8	38.1	30.0	72.5	26.9	1.2
GistMem	84.4	11.7	43.1	9.5	0.0	82.5	11.7	43.6	8.4	0.0	83.0	13.4	49.6	11.7	0.5
ReadAgent-P: Lookup 1 snippet	55.1	31.1	70.1	26.8	2.0	54.1	34.5	74.1	28.2	2.3	55.2	36.1	75.6	33.0	2.0
ReadAgent-P: Lookup 1-5 snippets	35.9	<u>33.7</u>	72.5	<u>29.2</u>	2.8	35.6	37.4	<u>75.1</u>	31.1	3.4	48.2	37.2	76.3	33.4	2.3
$\Delta(\text{Raw} \rightarrow \text{ReadAgent})$	–	+11.6	-4.2	+10.0	+1.6	–	+15.2	+2.8	+12.9	+1.7	–	+13.6	+0.7	+12.5	+1.3
$\Delta(\text{MindAct} \rightarrow \text{ReadAgent})$	–	+3.9	+10.6	+4.8	+1.6	–	+8.6	+15.5	+9.5	+2.8	–	+7.3	+15.9	+8.9	+1.0

Table 9: Web navigation performance on Mind2Web Deng et al. (2023). * marks models that are trained supervisedly for the web domain. GistMem and ReadAgent results are all also based on PaLM 2-L. We evaluate the performance in element accuracy (Ele. Acc), operation F1 (Op. F1), step success rate (Step SR), and episode success rate (SR). We also measure the compression rate (CR). The best performance across all the baselines is **bolded**, and the best across the approaches using PaLM 2-L is underlined. ReadAgent achieves consistently better performance than using raw HTML inputs (PaLM 2-L), retrieval methods, and MindAct (PaLM 2-L) with a trained Rank LM for HTML snippet retrieval.

the compression rate to drop substantially. In spite of those issues, the ReadAgent results give real gains over using the full context. This indicates that even the truncated gists and retrieved pages are more informative than the truncated raw HTML when using an LLM with a small context length.



Threshold	Raw	Gist
4096 Tokens	14.2%	88.6%
8192 Tokens	51.5%	97.4%
16384 Tokens	79.1%	100%
50th Percentile Tokens	8018	989
90th Percentile Tokens	25337	3596
95th Percentile Tokens	35779	5741
99th Percentile Tokens	55642	12569

Figure 9: **(Left)** Histogram of raw HTML and gist tokens in the Mind2Web cross-website split. Most of the input gists require fewer than 8K tokens. **(Right)** Statistics of token counts of raw HTML and gists.

E PROMPT DESIGN

We discuss our prompt design in this section to clarify some of our design decisions. In most cases, the exact phrasing of the prompt has negligible impacts on the outcome. For example, saying ”You don’t need to answer the question yet.” versus ”DO NOT answer the question in your response.” in the look-up prompts did not lead to significant differences in the results and they can be used interchangeably.

The exact prompts that we use for each dataset can be found on <https://read-agent.github.io/>.

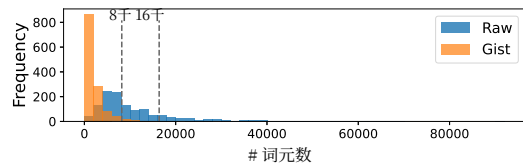
Pagination Prompt In the QuALITY and QMSum experiments where the target page length is short and the `max_words` threshold is low, we tried including the previous page in the pagination prompt (Section 3.1), as we thought it could be helpful to have more surrounding context. However, it appears that it only benefits our QMSum experiments, compared to not including the previous page. We will leave this to future studies.

Look-up Prompt On QuALITY, we found adding ”Take a deep breath and tell me: Which page(s) would you like to read again?” at the end of the look-up prompt improves response quality, when using PaLM 2-L and GPT-3.5. This is similar to what Yang et al. (2023) found.

	跨任务					跨网站					跨领域				
	CR	元素准确率	操作F1分数	步骤成功率	SR	CR	元素准确率	操作F1分数	步骤成功率	SR	CR	元素准确率	操作F1分数	步骤成功率	SR
使用监督训练的RankLM															
MindAct(GPT-3.5 + Rank LM*)	–	20.3	56.6	17.4	0.8	–	19.3	48.8	16.2	0.6	–	21.6	52.8	18.6	1.0
MindAct(GPT-4 + Rank LM*)	–	41.6	60.6	36.2	2.0	–	35.8	51.1	30.1	2.0	–	37.1	46.5	26.4	2.0
MindAct(PaLM 2-L + Rank LM*)	–	29.8	61.9	24.4	1.2	–	28.8	59.6	21.6	0.6	–	29.9	60.4	24.5	1.3
无训练															
(PaLM 2-L)															
+原始HTML	0.0	22.1	<u>76.7</u>	19.2	1.2	0.0	22.2	72.3	18.2	1.7	0.0	23.6	75.6	20.9	1.0
+BM25 检索 (Top-1)	43.7	16.3	61.7	14.2	0.4	49.7	17.8	60.8	15.2	0.0	51.6	17.3	60.4	15.9	0.0
+BM25 检索 (Top-5)	19.5	25.9	70.4	22.4	2.0	17.6	29.5	71.8	23.1	1.7	19.2	27.6	71.1	24.4	1.0
+神经检索 (Top-1)	74.4	14.6	55.5	11.7	0.4	87.9	18.0	55.8	14.0	0.0	82.8	16.4	60.3	14.2	0.1
+神经检索 (Top-5)	32.4	26.4	71.9	22.6	0.8	37.2	26.7	69.1	22.3	2.8	38.1	30.0	72.5	26.9	1.2
GistMem	84.4	11.7	43.1	9.5	0.0	82.5	11.7	43.6	8.4	0.0	83.0	13.4	49.6	11.7	0.5
ReadAgent-P: 查找 1 个片段	55.1	31.1	70.1	26.8	2.0	54.1	34.5	74.1	28.2	2.3	55.2	36.1	75.6	33.0	2.0
ReadAgent-P: 查找 1-5 个片段	35.9	<u>33.7</u>	72.5	<u>29.2</u>	2.8	35.6	37.4	<u>75.1</u>	31.1	3.4	48.2	37.2	76.3	33.4	2.3
$\Delta(\text{原始} \rightarrow \text{ReadAgent})$	–	+11.6	-4.2	+10.0	+1.6	–	+15.2	+2.8	+12.9	+1.7	–	+13.6	+0.7	+12.5	+1.3
$\Delta(\text{MindAct} \rightarrow \text{ReadAgent})$	–	+3.9	+10.6	+4.8	+1.6	–	+8.6	+15.5	+9.5	+2.8	–	+7.3	+15.9	+8.9	+1.0

表9: Mind2Web Deng et al. (2023) 上的网络导航性能。 * 标记了针对网络领域进行监督训练的模型。GistMem和ReadAgent的结果同样基于PaLM 2-L。我们评估了元素准确率 (Ele. Acc)、操作F1分数 (Op. F1)、步骤成功率 (Step SR) 和回合成功率 (SR) 的性能。我们还测量了压缩率 (CR)。所有基线中的最佳性能用**粗体**标出, 使用PaLM 2-L的方法中的最佳性能用下划线标出。ReadAgent在性能上持续优于使用原始HTML输入 (PaLM 2-L)、检索方法以及使用训练过的Rank LM进行HTML片段检索的MindAct (PaLM 2-L)。

压缩率大幅下降。尽管存在这些问题, ReadAgent的结果相比使用完整上下文仍带来了实际的增益。这表明, 在使用上下文长度较小的大语言模型时, 即使是截断后的要点和检索到的页面, 也比截断后的原始HTML更具信息量。



阈值	Raw	Gist
4096 词元	14.2%	88.6%
8192 词元	51.5%	97.4%
16384 词元	79.1%	100%
第50百分位标记数	8018	989
第90百分位标记数	25337	3596
第95百分位标记数	35779	5741
第99百分位标记数	55642	12569

图9: **(左)** Mind2Web跨网站分割中原始HTML和要点标记的直方图。大多数输入要点所需的标记少于8K令牌。**(右)** 原始HTML和要点的标记数量统计。

E 提示设计

我们在本节讨论我们的提示设计, 以阐明我们的一些设计决策。在大多数情况下, 提示的确切措辞对结果的影响微乎其微。例如, 在查找提示中说 ”你暂时不需要回答问题。” 与 “在你的回复中不要回答问题。” 并未导致结果的显著差异, 它们可以互换使用。

我们在每个数据集上使用的确切提示可在 <https://read-agent.github.io/> 找到。

分页提示 在QuALITY和QMSum实验中, 目标页面长度较短且最大词数阈值较低, 我们尝试在分页提示中包含前一页 (第3.1节), 因为我们认为拥有更多周围上下文可能有所帮助。然而, 与不包含前一页相比, 这似乎只对我们的QMSum实验有益。我们将把这个问题留给未来的研究。

查找提示 在QuALITY上, 我们发现当使用PaLM 2-L和GPT-3.5时, 在查找提示的末尾添加 “深呼吸一下然后告诉我: 你想重读哪一 (些) 页面?” 能提高回答质量。这与Yang等人 (2023) 的发现类似。

Response Prompt For tasks that have free-form answers (QMSum and NarrativeQA), we explicitly ask the model to provide short and concise answers as follows. This helps mitigate the problem of verbose answers hurting ROUGE scores as we noted in Section 4.5.

Example Response Prompt for Free-form Answers

{GISTS AND EXPANDED PAGES}
Question: {QUESTION}
Answer the question based on the above passage and retrieved pages. Your answer should be short and concise.

LLM Rater Prompts In Section 4.1, we introduce Strict LLM Rater Prompt for exact matches and Permissive LLM Rater Prompt for partial matches. The main reason for this design is that measuring exact matches can be overly strict as it disqualifies model answers with additional details. For example, assuming the reference answer is “the British” and the model answer is “the British army” (correct but with more details), using LLMs with “Strict LLM Rater Prompt” will tell us that this is not an exact match. Empirically, we found these LLM ratings aligning well with our own judgements.

When referring to model’s responses in the LLM rater prompts, we choose to refer them with “John’s answer to the question was:” because we want to prompt the LLM to judge whether the model-generated answer matches with the reference answer from a more objective, third-person view. Prior work, such as Zheng et al. (2023), uses “Assistant” to refer to responses from different model calls. Here we instead choose a common name (John) that appears to be more natural.

F READAGENT VARIANTS

F.1 UNCONDITIONAL AND CONDITIONAL READAGENT

When working with a long text, it is possible that the user will know ahead of time what task is to be solved. In that case, conceivably the gisting step could include the task description in the prompt. In so doing, it is easy to imagine that the LLM could do a better job of compressing out information that is irrelevant to the task, thereby improving efficiency and reducing distraction. This approach would be *Conditional ReadAgent*. However, more generally, the task may not be known while preparing the gists, or it may be known that the gists need to be used for multiple different tasks, such as answering many questions about the text. Thus, by excluding the task in the gisting step, the LLM may produce more broadly useful gists, at the cost of reduced compression and increased distracting information. This setting would be *Unconditional ReadAgent*. We only explore the unconditional setting in this work, but we note that the conditional setting may be preferred in some situations.

F.2 READAGENT FOR SPECIFIC DOMAINS

Related to Appendix F.1, when applying ReadAgent to specific domains, it might be helpful to provide domain-specific instructions. For example, if we apply ReadAgent to understanding a programming library, it could be useful to provide more specific instruction to LLMs to extract abstract descriptions of things like purpose of the code, functionalities, important signatures of functions or classes from each file as gists.

F.3 ITERATIVE GISTING

For a very long event history, such as a conversation, we may consider further compression of the older memory with iterative gisting to allow having longer contexts, similar to older memories of humans being fuzzier. Though this is not in the scope of this work, it may be useful for applications such as assistant agents, where context lengths can grow arbitrarily long over time as the user interacts with the agent.

响应提示对于需要自由形式答案的任务（QMSum和NarrativeQA），我们明确要求模型提供简短扼要的答案，如下所示。这有助于缓解我们在第4.5节中指出的冗长答案损害ROUGE分数的问题。

自由形式答案的示例响应提示

{要点与扩展页面}问题: {问题}请根据以上段落和检索到的页面回答问题。你的答案应简短扼要。

LLM评分提示在第4.1节中，我们介绍了用于精确匹配的严格LLM评分提示和用于部分匹配的宽松LLM评分提示。这样设计的主要原因是，测量精确匹配可能过于严格，因为它会排除那些包含额外细节的模型答案。例如，假设参考答案是 “the British”，而模型答案是 “the British army”（正确但包含更多细节），使用带有 “严格LLM评分提示” 的大语言模型会告诉我们这不是精确匹配。根据经验，我们发现这些LLM评分与我们自己的判断非常吻合。

在LLM评分提示中提及模型的回答时，我们选择用 “John对该问题的回答是：” 来指代它们，因为我们希望提示大语言模型从一个更客观的第三人称视角来判断模型生成的答案是否与参考答案匹配。先前的工作，例如Zheng等人（2023），使用 “助手” 来指代不同模型调用的回答。这里我们选择了一个看起来更自然的常见名字（John）。

F READAGENT 变体

F.1 无条件与有条件 READAGENT

在处理长文本时，用户可能提前知道需要解决什么任务。在这种情况下，可以设想在要点提取步骤的提示中包含任务描述。这样做，很容易想象大语言模型能更好地压缩掉与任务无关的信息，从而提高效率并减少干扰。这种方法将是有条件*ReadAgent*。然而，更普遍的情况是，在准备要点时可能并不知道具体任务，或者已知要点需要用于多个不同的任务，例如回答关于文本的许多问题。因此，通过在要点提取步骤中排除任务信息，大语言模型可能会产生更广泛适用的要点，但代价是压缩率降低和干扰信息增加。这种设置将是无条件*ReadAgent*。我们在本文中仅探索无条件设置，但我们注意到，在某些情况下，有条件设置可能更受青睐。

F.2 面向特定领域的 ReadAgent

与附录 F.1 相关，当将 ReadAgent 应用于特定领域时，提供领域特定指令可能会有所帮助。例如，如果我们将 ReadAgent 应用于理解一个编程库，向大语言模型提供更具体的指令以从每个文件中提取诸如代码目的、功能、函数或类的重要签名等抽象描述作为要点，可能会很有用。

F.3 迭代摘要

对于一个非常长的事件历史，例如一段对话，我们可以考虑通过迭代摘要对旧记忆进行进一步压缩，以容纳更长的上下文，类似于人类的旧记忆会变得更模糊。虽然这不在本工作的范围内，但对于诸如助理代理之类的应用可能很有用，在这些应用中，随着用户与代理的交互，上下文长度会随时间任意增长。

G COMPARING READAGENT AND MEMWALKER

As discussed in Section 2, similar to our work, MemWalker Chen et al. (2023a) also reads long documents interactively like an agent through iterative prompting, instead of forcing LLMs to process everything at once. It first constructs a summary tree where the lowest-level leafs are segments of raw text, the second level nodes are summaries of text segments, and the higher levels are summaries of summaries. Given a task, it traverses the tree from the root to search for task-related information. We think there are a few reasons to prefer the ReadAgent approach over MemWalker.

First, the reliability is a concern. Having LLMs traverse summary tree may not be a reliable process. In our best-effort re-implementation of MemWalker with PaLM 2-L, it unsatisfyingly achieves 66.73% on QuALITY. To put that into perspective, using full raw content is 85.83%, ReadAgent-P (look up 1-5 pages) is 86.63%, ReadAgent-S (look up 1-6 pages) is 86.88%, and using BM25 Top-1 is 70.55%. Part of the performance difference is caused by a high search failure rate. 11.7% of the searches failed to finish after sufficient retries. This failure rate of our implementation is in a similar range to what the authors reported: 91.4% successes and 8.6% failures⁵. In contrast, the failure rate of ReadAgent is mostly 0%.

Second, the hierarchical summary structure makes it difficult to reason over related but distant information at the same granularity. There isn’t much detail preserved at the top levels of the hierarchy. For example, if the two most important text pieces are at the beginning and the end of a very long text, the essential information could be in the first and last leaf. As the agent traverses down to the first leaf, it could be difficult to go back up to the root and down to the last leaf.

The motivations of the two approaches are also different. MemWalker interacts with a summary tree and reasons over traversal trajectories, whereas ReadAgent interacts directly with documents and reasons over gist memories.

H PAGINATION HYPERPARAMETERS

Pagination Details As described in Section 3.1, `max_words` and `min_words` are two episode pagination hyperparameters. Table 10 gives their values for each of the experiments in Section 4.

Dataset	$\frac{\text{max_words}}{\text{min_words}}$
QuALITY	$\frac{600}{280}$
QMSum	$\frac{600}{280}$
NarrativeQA Gutenberg	$\frac{3000}{500}$
NarrativeQA movie scripts	$\frac{1000}{600}$

Table 10: Pagination hyperparameters.

I NARRATIVEQA ADDITIONAL DETAILS

Figures 10 and 11 show histograms of word counts for the two NarrativeQA subsets and their corresponding gist memories.

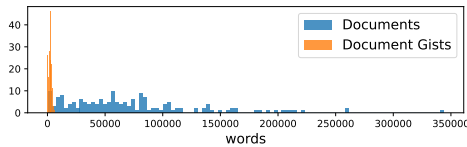


Figure 10: Histogram of NarrativeQA (Gutenberg) test set word counts for the original text and the gists.

⁵<https://openreview.net/forum?id=H5XZLeXWPS>

G 比较 READAGENT 与 MEMWALKER

如第2节所述，与我们的工作类似，MemWalker (Chen等人, 2023a) 也通过迭代提示，像代理一样交互式地阅读长文档，而不是强迫大语言模型一次性处理所有内容。它首先构建一个摘要树，其中最低层的叶子是原始文本片段，第二层节点是文本片段的摘要，更高层则是摘要的摘要。给定一个任务，它会从根节点开始遍历树以搜索任务相关信息。我们认为，相较于 MemWalker，ReadAgent方法有几个更可取的理由。

首先，可靠性是一个问题。让大语言模型遍历摘要树可能不是一个可靠的过程。在我们使用 PaLM 2-L 对 MemWalker 进行的最佳努力复现中，其在 QuALITY 数据集上的表现仅为 66.73%，这并不令人满意。作为对比，使用完整原始内容为 85.83%，ReadAgent-P（查阅 1-5 个页面）为 86.63%，ReadAgent-S（查阅 1-6 个页面）为 86.88%，而使用 BM25 Top-1 为 70.55%。部分性能差异是由较高的搜索失败率造成的。11.7% 的搜索在充分重试后仍未能完成。我们实现的这个失败率与原作者报告的范围相似：91.4% 成功，8.6% 失败⁵。相比之下，ReadAgent 的失败率大多为 0%。

其次，分层摘要结构使得在同一粒度上对相关但距离较远的信息进行推理变得困难。在层次结构的顶层，保留的细节信息并不多。例如，如果两个最重要的文本片段位于一篇极长文本的开头和结尾，那么关键信息可能分别位于第一个和最后一个叶子节点中。当智能体向下遍历到第一个叶子节点时，它可能很难再返回到根节点，然后向下遍历到最后一个叶子节点。

两种方法的动机也不同。MemWalker 与摘要树交互，并对遍历轨迹进行推理；而 ReadAgent 则直接与文档交互，并对要点记忆进行推理。

H 分页超参数

分页详情如第3.1节所述，最大词数和最小词数是两个片段分页超参数。表10给出了第4节中各项实验所采用的参数值。

数据集	$\frac{\text{最大词数}}{\text{最小词数}}$
QuALITY	$\frac{600}{280}$
QMSum	$\frac{600}{280}$
NarrativeQA Gutenberg	$\frac{3000}{500}$
NarrativeQA 电影脚本	$\frac{1000}{600}$

表 10：分页超参数。

I NARRATIVEQA 补充细节

图10和图11展示了两个NarrativeQA子集及其对应要点记忆的词数直方图。

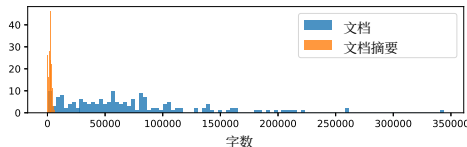


图10：NarrativeQA (Gutenberg) 测试集中原始文本与要点的词数分布直方图。

⁵<https://openreview.net/forum?id=H5XZLeXWPS>

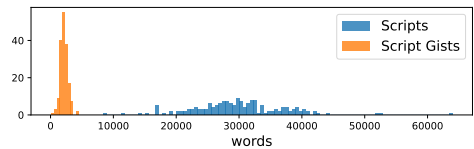


Figure 11: Histogram of NarrativeQA (movie) test set word counts for the original text and the gists.

Context Length Control As the NarrativeQA Gutenberg texts can be very long, the corresponding gists can sometimes exceed the context length. For those exceptionally long texts, we ask the LLM to go through the pages and think whether it makes sense to merge pages iteratively with the following prompt, and then re-gist the new set of pages. In so doing, we are able to increase the average page size and thus the compression rate (Figure 12).

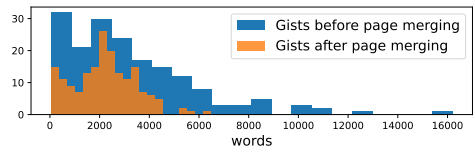


Figure 12: Histogram of NarrativeQA (Gutenberg) test set gists before and after page merging on the exceptionally long texts.

Example NarrativeQA Gutenberg Page Merging Prompt

Given Page 1 and Page 2, please tell me whether Page 2 starts a new chapter/section/book that is different from what’s in Page 1.
Please answer with yes, no, or not sure.
Page 1:
{PREVIOUS PAGE TEXT}
Page 2:
{CURRENT PAGE TEXT}

The gists and pages can both be long for NarrativeQA. Thus, in the interactive look-up step of ReadAgent-P, we prevent the retrieved pages from exceeding the context length by asking the model to sort the pages by importance with the prompt below and iteratively detecting whether adding any pages could go beyond the context window. For ReadAgent-S, we do a similar check to decide whether to early-stop the sequential look-up.

Example Parallel Lookup Prompt (ReadAgent-P) for NarrativeQA

The following text is what you remember from reading an article and a question related to it.
You may read 1, 2 or 3 page(s) of the article again to refresh your memory to prepare yourself for the question.
Please respond with which page(s) you would like to read in the order of importance, beginning with the most important page number.
For example, if you only need to read Page 8, respond with “I want to look up Page [8] to ...”.
If you would like to read Page 12 and 7, respond with “I want to look up Page [12, 7] to ...”.
If you would like to read Page 15, 2 and 3, respond with “I want to look up Page [15, 2, 3] to ...”.
DO NOT select more pages if you don’t need to.
You don’t need to answer the question yet.
Text:
{GIST MEMORY}
Question:
{QUESTION}

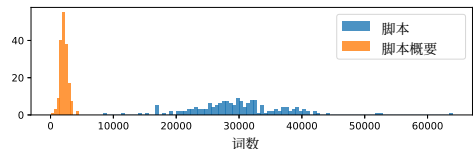


图 11: NarrativeQA（电影）测试集中原始文本与要点的词数分布直方图。

上下文长度控制由于NarrativeQA Gutenberg文本可能非常长，对应的要点有时会超出上下文长度。对于那些特别长的文本，我们会要求大语言模型浏览各个页面，并通过以下提示词思考是否应该迭代地合并页面，然后重新为新的一组页面生成要点。通过这种方式，我们能够增加平均页面大小，从而提高压缩率（图12）。

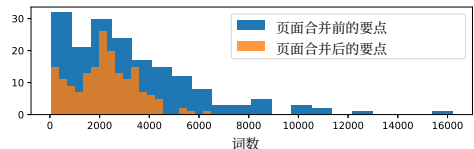


图 12: 针对超长文本，NarrativeQA (Gutenberg) 测试集要点在页面合并前后的分布直方图。

NarrativeQA Gutenberg 页面合并提示示例

给定页面 1 和页面 2，请告诉我页面 2 是否开启了一个与页面 1 内容不同的新章节/新部分/新书。

请用“是”、“否”或“不确定”来回答。
第 1 页:
{前一页文本}
第 2 页:
{当前页文本}

对于 NarrativeQA，要点和页面都可能很长。因此，在 ReadAgent-P 的交互式查找步骤中，我们通过要求模型根据以下提示按重要性对页面进行排序，并迭代检测添加任何页面是否会超出上下文窗口，来防止检索到的页面超出上下文长度。对于 ReadAgent-S，我们进行类似的检查，以决定是否提前停止顺序查找。

NarrativeQA 的示例并行查找提示 (ReadAgent-P)

以下文本是您阅读一篇文章及相关问题后记住的内容。您可以重新阅读文章的1、2或3个页面来刷新记忆，为回答问题做准备。请按重要性顺序，从最重要的页码开始，回复您希望阅读哪个（些）页面。例如，如果您只需要阅读第8页，请回复 “I want to look up Page [8] to ...”。如果您希望阅读第12页和第7页，请回复 “I want to look up Page [12, 7] to ...”。如果您希望阅读第15页、第2页和第3页，请回复 “I want to look up Page [15, 2, 3] to ...”。如果不需要，请勿选择更多页面。您暂时无需回答问题。文本: {要点记忆}问题: {问题}

Method	CR (# LU)	LLM Rating-1	LLM Rating-2	ROUGE-1	ROUGE-2	ROUGE-L	Resp. Length
BM25 Retrieval							
Top-1	95.61% (1.00)	24.67% $\pm$ 0.44	66.90% $\pm$ 0.87	28.81 $\pm$ 0.13	8.14 $\pm$ 0.15	19.62 $\pm$ 0.18	48.15 $\pm$ 0.18
Top-2	91.32% (2.00)	31.79% $\pm$ 1.31	79.95% $\pm$ 0.67	30.89 $\pm$ 0.13	9.14 $\pm$ 0.05	20.67 $\pm$ 0.09	53.91 $\pm$ 0.64
Top-3	87.25% (3.00)	33.45% $\pm$ 0.00	83.63% $\pm$ 1.05	31.39 $\pm$ 0.23	9.11 $\pm$ 0.05	21.03 $\pm$ 0.03	55.15 $\pm$ 0.61
Top-4	82.86% (4.00)	37.72% $\pm$ 1.05	86.12% $\pm$ 0.50	31.71 $\pm$ 0.09	9.35 $\pm$ 0.13	21.26 $\pm$ 0.12	58.21 $\pm$ 0.37
Top-5	78.79% (5.00)	39.38% $\pm$ 1.02	86.60% $\pm$ 0.44	32.66 $\pm$ 0.04	9.98 $\pm$ 0.10	21.86 $\pm$ 0.05	59.20 $\pm$ 1.05
Top-6	74.62% (6.00)	40.45% $\pm$ 0.89	90.98% $\pm$ 0.34	32.56 $\pm$ 0.03	9.78 $\pm$ 0.03	21.64 $\pm$ 0.09	60.40 $\pm$ 1.28
Neural Retrieval with Gemini API							
Top-1	95.80% (1.00)	27.05% $\pm$ 0.50	67.97% $\pm$ 1.74	28.71 $\pm$ 0.12	7.98 $\pm$ 0.04	19.59 $\pm$ 0.04	49.76 $\pm$ 0.78
Top-2	91.62% (2.00)	35.35% $\pm$ 0.44	80.07% $\pm$ 0.00	31.65 $\pm$ 0.18	9.59 $\pm$ 0.11	21.29 $\pm$ 0.11	56.19 $\pm$ 0.76
Top-3	87.39% (3.00)	35.71% $\pm$ 1.37	88.49% $\pm$ 0.34	32.33 $\pm$ 0.17	9.84 $\pm$ 0.07	21.54 $\pm$ 0.13	59.19 $\pm$ 0.96
Top-4	83.28% (4.00)	39.62% $\pm$ 0.17	90.15% $\pm$ 0.34	32.31 $\pm$ 0.21	9.69 $\pm$ 0.15	21.65 $\pm$ 0.15	59.86 $\pm$ 0.11
Top-5	79.33% (5.00)	44.01% $\pm$ 0.84	91.22% $\pm$ 0.34	32.33 $\pm$ 0.24	9.84 $\pm$ 0.21	21.67 $\pm$ 0.19	61.53 $\pm$ 0.35
Top-6	75.35% (6.00)	44.60% $\pm$ 0.89	92.65% $\pm$ 0.17	32.55 $\pm$ 0.08	9.75 $\pm$ 0.21	21.39 $\pm$ 0.13	61.29 $\pm$ 0.46
Truncated Raw Content							
First 6k words	31.51% (0.00)	13.17% $\pm$ 1.05	47.81% $\pm$ 5.90	24.15 $\pm$ 1.42	4.89 $\pm$ 0.57	16.27 $\pm$ 0.96	61.43 $\pm$ 3.53
Last 6k words	33.80% (0.00)	13.76% $\pm$ 0.84	43.42% $\pm$ 0.00	22.90 $\pm$ 0.10	4.35 $\pm$ 0.04	15.69 $\pm$ 0.03	52.47 $\pm$ 0.39
GistMem	82.81% (0.00)	44.96% $\pm$ 0.44	91.93% $\pm$ 0.73	31.20 $\pm$ 0.17	9.02 $\pm$ 0.09	20.60 $\pm$ 0.14	65.84 $\pm$ 0.87
ReadAgent-P							
Look up 1 pg	79.37% (0.98)	44.84% $\pm$ 0.00	92.29% $\pm$ 0.34	31.46 $\pm$ 0.12	9.09 $\pm$ 0.11	20.63 $\pm$ 0.05	66.74 $\pm$ 0.74
Look up 1-2 pgs	77.00% (1.72)	43.42% $\pm$ 1.01	92.88% $\pm$ 1.05	31.77 $\pm$ 0.16	9.11 $\pm$ 0.12	20.70 $\pm$ 0.08	65.55 $\pm$ 0.28
Look up 1-3 pgs	74.85% (2.46)	44.37% $\pm$ 1.21	91.22% $\pm$ 0.44	31.89 $\pm$ 0.06	8.98 $\pm$ 0.13	20.70 $\pm$ 0.09	66.06 $\pm$ 1.63
Look up 1-4 pgs	73.26% (3.02)	44.13% $\pm$ 0.50	90.51% $\pm$ 0.44	31.87 $\pm$ 0.07	9.12 $\pm$ 0.06	20.77 $\pm$ 0.01	66.44 $\pm$ 0.74
Look up 1-5 pgs	72.01% (3.44)	43.42% $\pm$ 1.45	91.22% $\pm$ 0.60	31.80 $\pm$ 0.16	9.03 $\pm$ 0.07	20.64 $\pm$ 0.03	66.48 $\pm$ 0.39
Look up 1-6 pgs	70.65% (3.89)	42.70% $\pm$ 1.54	90.51% $\pm$ 0.73	31.74 $\pm$ 0.09	8.90 $\pm$ 0.09	20.66 $\pm$ 0.16	66.24 $\pm$ 1.14
ReadAgent-S 1-6 pgs	70.75% (3.42)	49.58 $\pm$ 0.44	93.83 $\pm$ 0.34	32.88 $\pm$ 0.15	9.98 $\pm$ 0.06	21.50 $\pm$ 0.04	67.86 $\pm$ 0.11

Table 11: **QMSum test** results (PaLM 2-L) means and standard deviations across 3 runs. 35 articles and 281 questions. **Bold methods** are this work. **Bold** values are the best; ***bold italics*** are ties for best. **CR** is the compression rate. **# LU** is the number of lookups. **Resp. Length** is the length in words of the model’s final response. We omit standard deviations for CR and # LU for presentation purposes; they were all inconsequential.

J ADDITIONAL QMSUM RESULTS

Figure 13 shows the histogram of word counts on the QMSum training set and the corresponding gist memories. Table 11 shows the same results as Table 3, but on the QMSum test set.

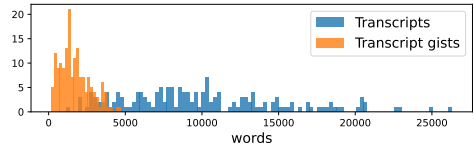


Figure 13: Histogram of QMSum word counts for the original transcripts and the gisted transcripts. The gisted transcripts are all less than 5,000 words, allowing them to entirely fit into the context windows of PaLM 2-L.

K AUTHOR CONTRIBUTIONS

Kuang-Huei Lee developed the initial working prototype, the method and the experiments on QuALITY and NarrativeQA, was a main writer of the manuscript, and led the project overall.

Xinyun Chen developed the method, the LLM rater, and experiments on NarrativeQA, and significantly contributed to manuscript writing.

Hiroki Furuta developed the web navigation experiments, and significantly contributed to manuscript writing.

John Canny contributed in the initial conceptualization, advised the project, and helped with manuscript editing.

Ian Fischer co-proposed the core idea, developed the method and experiments on QMSum, and was a main writer of the manuscript.

方法	压缩率 (# LU)	LLM评分-1	LLM评分-2	ROUGE-1	ROUGE-2	ROUGE-L	响应长度
BM25 检索							
Top-1	95.61% (1.00)	24.67% $\pm$ 0.44	66.90% $\pm$ 0.87	28.81 $\pm$ 0.13	8.14 $\pm$ 0.15	19.62 $\pm$ 0.18	48.15 $\pm$ 0.18
前两名	91.32% (2.00)	31.79% $\pm$ 1.31	79.95% $\pm$ 0.67	30.89 $\pm$ 0.13	9.14 $\pm$ 0.05	20.67 $\pm$ 0.09	53.91 $\pm$ 0.64
Top-3	87.25% (3.00)	33.45% $\pm$ 0.00	83.63% $\pm$ 1.05	31.39 $\pm$ 0.23	9.11 $\pm$ 0.05	21.03 $\pm$ 0.03	55.15 $\pm$ 0.61
Top-4	82.86% (4.00)	37.72% $\pm$ 1.05	86.12% $\pm$ 0.50	31.71 $\pm$ 0.09	9.35 $\pm$ 0.13	21.26 $\pm$ 0.12	58.21 $\pm$ 0.37
前五	78.79% (5.00)	39.38% $\pm$ 1.02	86.60% $\pm$ 0.44	32.66 $\pm$ 0.04	9.98 $\pm$ 0.10	21.86 $\pm$ 0.05	59.20 $\pm$ 1.05
前六	74.62% (6.00)	40.45% $\pm$ 0.89	90.98% $\pm$ 0.34	32.56 $\pm$ 0.03	9.78 $\pm$ 0.03	21.64 $\pm$ 0.09	60.40 $\pm$ 1.28
使用Gemini API进行神经检索							
Top-1	95.80% (1.00)	27.05% $\pm$ 0.50	67.97% $\pm$ 1.74	28.71 $\pm$ 0.12	7.98 $\pm$ 0.04	19.59 $\pm$ 0.04	49.76 $\pm$ 0.78
前两名	91.62% (2.00)	35.35% $\pm$ 0.44	80.07% $\pm$ 0.00	31.65 $\pm$ 0.18	9.59 $\pm$ 0.11	21.29 $\pm$ 0.11	56.19 $\pm$ 0.76
Top-3	87.39% (3.00)	35.71% $\pm$ 1.37	88.49% $\pm$ 0.34	32.33 $\pm$ 0.17	9.84 $\pm$ 0.07	21.54 $\pm$ 0.13	59.19 $\pm$ 0.96
Top-4	83.28% (4.00)	39.62% $\pm$ 0.17	90.15% $\pm$ 0.34	32.31 $\pm$ 0.21	9.69 $\pm$ 0.15	21.65 $\pm$ 0.15	59.86 $\pm$ 0.11
前五	79.33% (5.00)	44.01% $\pm$ 0.84	91.22% $\pm$ 0.34	32.33 $\pm$ 0.24	9.84 $\pm$ 0.21	21.67 $\pm$ 0.19	61.53 $\pm$ 0.35
前6名	75.35% (6.00)	44.60% $\pm$ 0.89	92.65% $\pm$ 0.17	32.55 $\pm$ 0.08	9.75 $\pm$ 0.21	21.39 $\pm$ 0.13	61.29 $\pm$ 0.46
截断的原始内容							
前6千词	31.51% (0.00)	13.17% $\pm$ 1.05	47.81% $\pm$ 5.90	24.15 $\pm$ 1.42	4.89 $\pm$ 0.57	16.27 $\pm$ 0.96	61.43 $\pm$ 3.53
最后6千词	33.80% (0.00)	13.76% $\pm$ 0.84	43.42% $\pm$ 0.00	22.90 $\pm$ 0.10	4.35 $\pm$ 0.04	15.69 $\pm$ 0.03	52.47 $\pm$ 0.39
GistMem	82.81% (0.00)	44.96% $\pm$ 0.44	91.93% $\pm$ 0.73	31.20 $\pm$ 0.17	9.02 $\pm$ 0.09	20.60 $\pm$ 0.14	65.84 $\pm$ 0.87
ReadAgent-P							
查找1页	79.37% (0.98)	44.84% $\pm$ 0.00	92.29% $\pm$ 0.34	31.46 $\pm$ 0.12	9.09 $\pm$ 0.11	20.63 $\pm$ 0.05	66.74 $\pm$ 0.74
查找1-2页	77.00% (1.72)	43.42% $\pm$ 1.01	92.88% $\pm$ 1.05	31.77 $\pm$ 0.16	9.11 $\pm$ 0.12	20.70 $\pm$ 0.08	65.55 $\pm$ 0.28
查找1-3页	74.85% (2.46)	44.37% $\pm$ 1.21	91.22% $\pm$ 0.44	31.89 $\pm$ 0.06	8.98 $\pm$ 0.13	20.70 $\pm$ 0.09	66.06 $\pm$ 1.63
查找1-4页	73.26% (3.02)	44.13% $\pm$ 0.50	90.51% $\pm$ 0.44	31.87 $\pm$ 0.07	9.12 $\pm$ 0.06	20.77 $\pm$ 0.01	66.44 $\pm$ 0.74
查阅 1-5 页	72.01% (3.44)	43.42% $\pm$ 1.45	91.22% $\pm$ 0.60	31.80 $\pm$ 0.16	9.03 $\pm$ 0.07	20.64 $\pm$ 0.03	66.48 $\pm$ 0.39
查阅 1-6 页	70.65% (3.89)	42.70% $\pm$ 1.54	90.51% $\pm$ 0.73	31.74 $\pm$ 0.09	8.90 $\pm$ 0.09	20.66 $\pm$ 0.16	66.24 $\pm$ 1.14
ReadAgent-S1-6 页	70.75% (3.42)	49.58 $\pm$ 0.44	93.83 $\pm$ 0.34	32.88 $\pm$ 0.15	9.98 $\pm$ 0.06	21.50 $\pm$ 0.04	67.86 $\pm$ 0.11

表11: **QMSum测试结果**（PaLM 2-L）为3次运行的平均值和标准差。共35篇文章和281个问题。**加粗方法**为本工作。**加粗值**为最佳；加粗斜体表示并列最佳。**压缩率**为压缩率。**#查找次数**为查找次数。**响应长度**为模型最终响应的单词长度。为呈现清晰，我们省略了压缩率和查找次数的标准差；它们均微不足道。

J 补充的 QMSum 结果

图13显示了 QMSum 训练集上词数的直方图及其对应的要点记忆。表11展示了与表3相同的结果，但基于 QMSum 测试集。

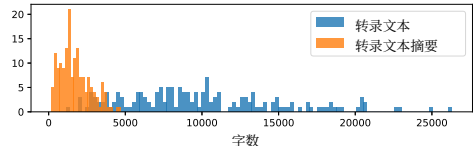


图13：原始转录本与概要转录本的 QMSum 词数直方图。所有概要转录本均少于 5,000 词，使其能完全放入 PaLM 2-L 的上下文窗口中。

K 作者贡献

Kuang-Huei Lee 开发了初始工作原型、方法以及在 QuALITY 和 NarrativeQA 上的实验，是论文的主要撰写者，并总体领导了该项目。

Xinyun Chen 开发了方法、LLM 评分器以及在 NarrativeQA 上的实验，并对论文撰写做出了重要贡献。

HirokiFuruta 开发了网络导航实验，并对论文撰写做出了重要贡献。

John Canny 参与了最初的构思，指导了项目，并协助了论文编辑。

IanFischer 共同提出了核心想法，开发了QMSum上的方法与实验，并且是论文的主要撰写者之一。